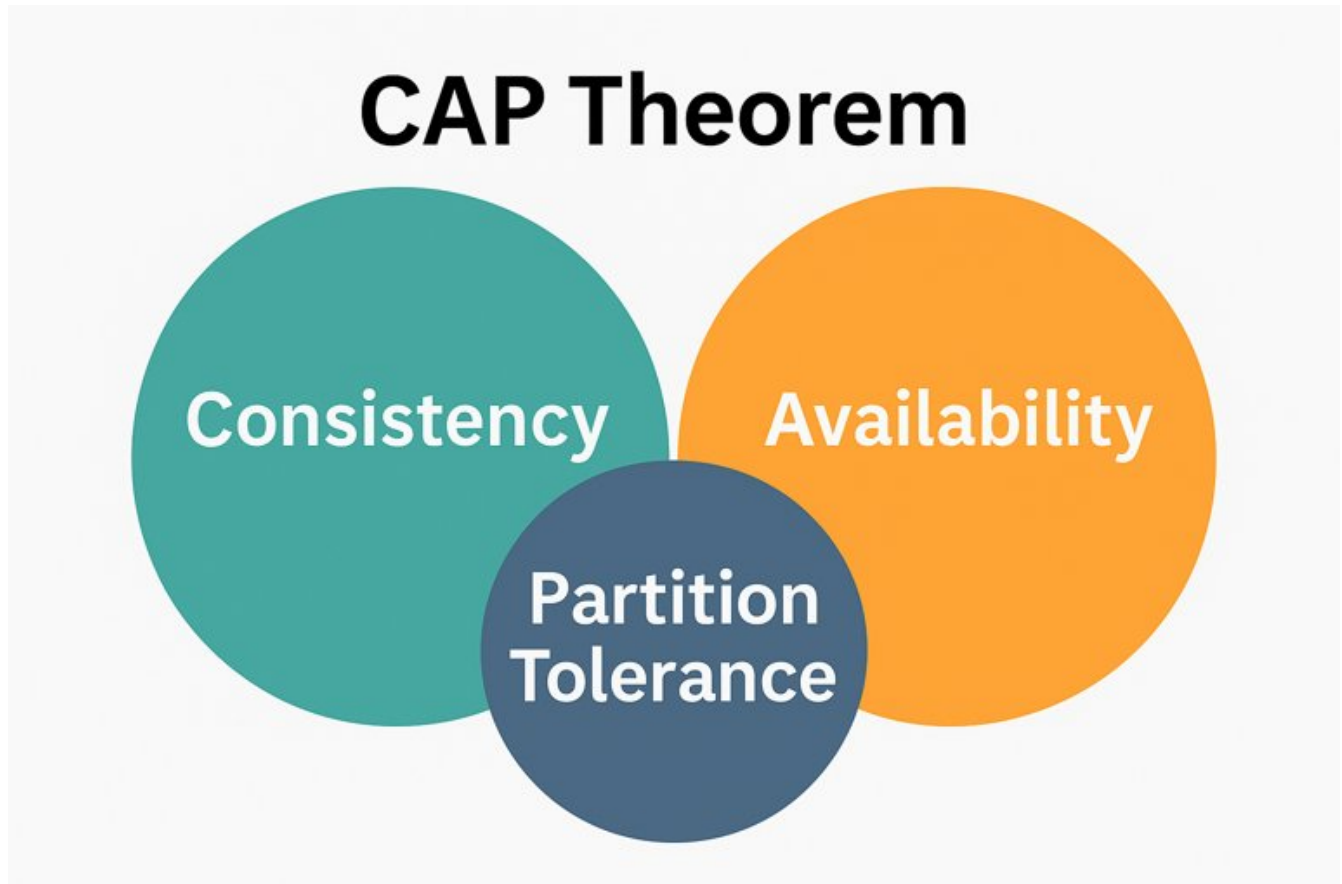


JAVA Developer Notes

Kovteba Dima <kovteba@gmail.com> v1.0, 2012-02-10

1. Architecture

1.1. CAP Theory



CAP-теорія — це фундаментальна концепція, яка описує обмеження при проектуванні розподілених систем. Вона була сформульована в 1999 році Еріком Брюером (Eric Brewer), а у 2002 році [математично доведена](#). Ця теорія особливо важлива при створенні систем Service Discovery, баз даних, кластерів та хмарних інфраструктур.

Назва CAP походить від трьох властивостей, які розподілена система може мати:

- **C (Consistency / Узгодженість)** — всі вузли системи бачать одні й ті ж дані одночасно. Тобто, після запису до системи, будь-яке подальше читання повертає саме це нове значення.
- **A (Availability / Доступність)** — кожен запит до системи отримує відповідь, навіть якщо частина вузлів недоступна.
- **P (Partition tolerance / Толерантність до розділення)** — система продовжує працювати навіть у випадку втрати зв'язку між частинами кластеру (мережевого поділу).

Принцип вибору 2 з 3

CAP-теорема стверджує: у розподіленій системі, яка піддається мережевим розділенням (а це неминуче в реальному світі), можна одночасно гарантувати лише дві з трьох властивостей — C, A, P.

Це означає, що кожна система повинна робити вибір між:

- CA (узгодженість + доступність) — але без толерантності до мережових розділень;
- CP (узгодженість + толерантність) — доступність може постраждати при збої;
- AP (доступність + толерантність) — жертвується повна консистентність.

Комбінація	Що це означає	Приклади систем
C + P	Дані узгоджені, але при розділенні кластеру частина запитів буде блокуватись	Etcd, Zookeeper, Consul
A + P	Система завжди відповідає, але може повертати застарілі/несинхронні дані	Cassandra, DynamoDB
C + A	Можлива тільки у централізованих системах або без мережових проблем	СУБД без реплікації

Для досягнення узгодженості © у розподілених системах типу CP використовуються механізми **кворуму** та **консенсусу**. Де **консенсус** — це процес, за допомогою якого вузли в системі погоджуються на певний стан або значення. **Кворум** — це мінімальна кількість вузлів, які повинні погодитися для прийняття рішення, тобто як що у кластері з 5 вузлів, кворум = 3 (тобто більшість). Якщо 3+ вузли згодні — система дозволяє запис. Інакше — ні. Наприклад Etcd, Consul та Zookeeper реалізують це через алгоритми **консенсусу**: **Raft**, **Zab** або подібні. Тобто, якщо кластер Consul має 5 вузлів, він може втратити 2 вузли і все ще залишатися узгодженим (3 з 5 = кворум). Але якщо втратить 3 — кворум буде втрачено, і нові записи блокуються. Залишаються тільки читання з кешу або останньої доступної копії (залежить від реалізації).

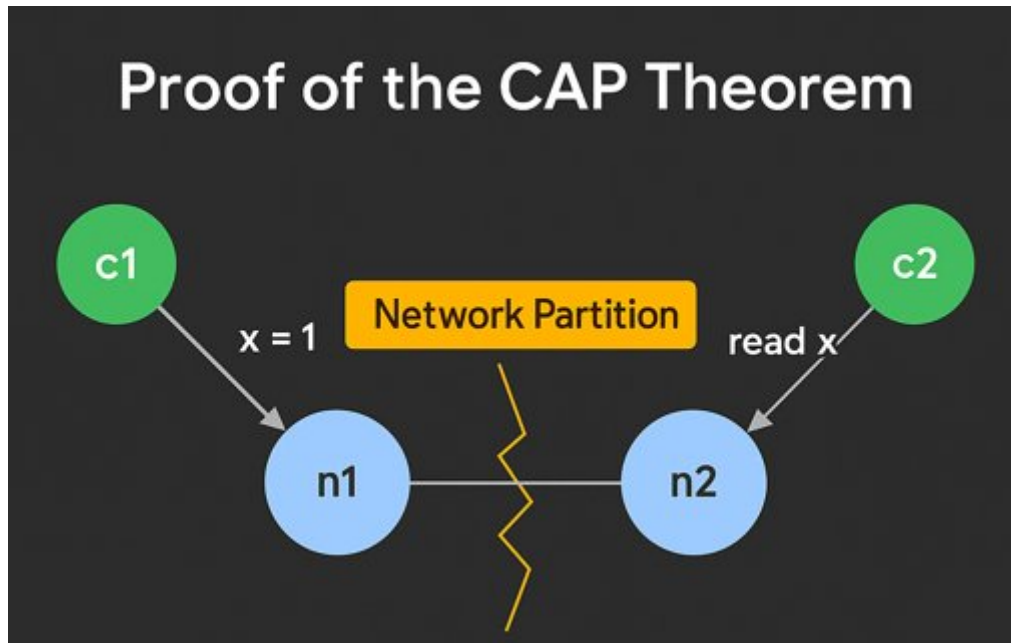
Підходи у продакшені:

Вимоги	Тип системи	Приклади
Уникати конфліктів і неточних даних	CP	Etcd, Zookeeper, Consul
Гарантувати швидкий відгук за будь-яких умов	AP	DynamoDB, Cassandra, DNS
Локальна централізована система	CA (умовно)	PostgreSQL, MySQL

Як висновок, **CAP-теорія** — не просто теоретична концепція, а практичний інструмент для розробників розподілених систем. Розуміння компромісів між узгодженістю, доступністю та толерантністю до розділення допомагає створювати надійні, масштабовані та ефективні

системи, які відповідають вимогам сучасних застосунків.

1.2. CAP Theory proof



- 1999 — Ерік Брюер (Eric Brewer) сформулював гіпотезу CAP на конференції PODC (Principles of Distributed Computing);
- 2002 — гіпотеза була математично доведена Сетом Гілбертом (Seth Gilbert) і Ненсі Лінч (Nancy Lynch) з MIT у статті:

Gilbert, S., & Lynch, N. (2002). Brewer's conjecture and the feasibility of consistent, available, partition-tolerant web services. ACM SIGACT News, 33(2), 51–59.

Доказ

1. Є два вузли $n1$ і $n2$, які обслуговують клієнтів $c1$ і $c2$ відповідно. В обох вузлів є копія деякого значення x (наприклад, баланс рахунку).
2. Відбувається розділення мережі (partition): $n1$ і $n2$ більше не можуть обмінюватися повідомленнями. Обидва вузли залишаються живими і доступними для клієнтів.
3. Клієнт $c1$ надсилає до $n1$ операцію запису: $x = 1$ (наприклад, змінити баланс на 1).
4. Одночасно клієнт $c2$ надсилає до $n2$ операцію читання: $get(x)$ — дізнатися поточне значення x .

Суперечність:

1. Якщо $n2$ негайно повертає значення x , то воно буде застарілим (бо не знає, що $x = 1$ вже записане на $n1$). □ порушена узгодженість ©.
2. Якщо $n2$ не відповідає, чекаючи на підтвердження від $n1$, то клієнт $c2$ не отримає відповіді. □ порушена доступність (A).

Оскільки вузли не можуть комунікувати (через P), система не може забезпечити і C, і A

одночасно.



Це і є формальна суперечність, яка доводить CAP-теорему: в умовах Partition (P) неможливо мати одночасно Consistency © та Availability (A).

Висновок:

CAP — не просто гіпотеза чи «архітектурний вибір». Це математичне обмеження, яке неможливо обійти. Будь-яке реальне рішення — це компроміс між C, A, P залежно від пріоритетів системи.

1.3. Service Discovery

WHAT IS SERVICE DISCOVERY?



SERVICE



LOCATIONS



LOOKUP

Service Discovery — це механізм, який дозволяє автоматично знаходити та підключатися до сервісів у розподілених системах, таких як Docker Swarm або Kubernetes. Це особливо важливо в середовищах, де сервіси можуть динамічно змінювати IP-адресу, масштабуватися або переміщуватися між різними вузлами.

Наприклад, якщо у вас є сервіси *user-service*, *payment-service* та *email-service*, і сервіс *payment-service* хоче зв'язатися з *user-service*, то без Service Discovery потрібно вручну вказувати IP-адресу, наприклад: <http://192.168.0.15:8080>. Але якщо IP-адреса зміниться, *payment-service* більше не зможе знайти *user-service*. Завдяки Service Discovery *payment-service* може звернутися до *user-service* за його логічною назвою: <http://user-service:8080>, і система автоматично знайде правильну IP-адресу та порт.

Основні типи **Service Discovery**:

- [Client-side discovery](#);
- [Server-side discovery](#);
- [DNS-based discovery](#);
- [Key-Value Store-based discovery](#).

Client-side discovery (клієнтське виявлення) — це підхід до Service Discovery, при якому клієнт (тобто сервіс, який хоче знайти інший сервіс) самостійно звертається до сервісного реєстру, отримує список доступних інстансів потрібного сервісу і вибирає, до якого з них підключитися. У цьому випадку клієнт не покладається на проміжний проксі або шлюз для маршрутизації запитів. Прикладами систем, які використовують client-side discovery, є Netflix Eureka у поєднанні з Ribbon, а також клієнтські API Consul.

Як працює **Client-side discovery**:

1. Сервіс реєструє себе в сервісному реєстрі (наприклад, Eureka, Consul) при запуску, надаючи інформацію про своє розташування (IP-адресу, порт, метадані).
2. Коли клієнтський сервіс хоче звернутися до іншого сервісу, він запитує сервісний реєстр, щоб отримати список доступних інстансів цього сервісу.
3. Клієнт отримує список інстансів і використовує алгоритм балансування навантаження (наприклад, round-robin, random) для вибору одного з них.
4. Клієнт встановлює пряме з'єднання з обраним інстансом і виконує запит.
5. Якщо обраний інстанс недоступний, клієнт може повторити запит до сервісного реєстру, щоб отримати оновлений список інстансів і спробувати знову.

```
RestTemplate template = new RestTemplate();  
String url = "http://user-service"; // логічне ім'я
```

Spring Cloud з Eureka + Ribbon автоматично знайде IP.

Переваги та недоліки **Client-side discovery**:

Переваги:

- Менше навантаження на проміжні проксі, оскільки клієнт самостійно вибирає інстанс;
- Гнучкість у виборі алгоритмів балансування навантаження на стороні клієнта;
- Пряме з'єднання між клієнтом і сервісом, що може знизити затримки.

Недоліки:

- Кожен клієнт повинен мати логіку для взаємодії з сервісним реєстром, що ускладнює клієнтський код;
- Залежність від конкретного стека (наприклад, Eureka → Java only);
- Потрібно оновлювати всі клієнти при зміні логіки.

Server-side discovery (серверне виявлення) — це підхід до Service Discovery, при якому

клієнтський сервіс звертається до проміжного проксі або шлюзу, а цей проміжний компонент самостійно виконує запит до сервісного реєстру та переспрямовує запит до потрібного інстансу сервісу. У цьому випадку клієнт не має прямого доступу до сервісного реєстру і не відповідає за вибір інстансу. Прикладами систем, які використовують server-side discovery, є Kubernetes (через kube-proxy і DNS), Envoy та Istio.

Як працює **Server-side discovery**:

1. Сервіс реєструє себе в сервісному реєстрі (наприклад, Kubernetes API Server, Consul) при запуску, надаючи інформацію про своє розташування (IP-адресу, порт, метадані).
2. Клієнтський сервіс звертається до проміжного проксі або шлюзу, вказуючи логічне ім'я потрібного сервісу (наприклад, через DNS-ім'я або URL).
3. Проксі або шлюз запитує сервісний реєстр, щоб отримати список доступних інстансів цього сервісу.
4. Проксі або шлюз використовує алгоритм балансування навантаження (наприклад, round-robin, random) для вибору одного з інстансів.
5. Проксі або шлюз встановлює з'єднання з обраним інстансом і переспрямовує запит від клієнта.
6. Якщо обраний інстанс недоступний, проксі або шлюз може повторити запит до сервісного реєстру, щоб отримати оновлений список інстансів і спробувати знову.

```
RestTemplate template = new RestTemplate();  
String url = "http://gateway/user-service"; // через API Gateway
```

API Gateway (наприклад, Zuul, Kong) або сервісна сітка (наприклад, Istio) автоматично знайде IP.

Переваги та недоліки **Server-side discovery**:

Переваги:

- Спрощення клієнтського коду, оскільки клієнт не потребує логіки для взаємодії з сервісним реєстром;
- Централізоване управління маршрутизацією та балансуванням навантаження на стороні проксі або шлюзу;
- Легше впроваджувати політики безпеки, моніторинг і логування на рівні проксі або шлюзу.

Недоліки:

- Додаткове навантаження на проміжні проксі або шлюзи, що може стати вузьким місцем у системі;
- Затримки через додатковий рівень маршрутизації між клієнтом і сервісом;
- Залежність від надійності проміжного компонента (проксі або шлюзу).

DNS-based discovery (виявлення на основі DNS) — це підхід до Service Discovery, при якому сервіси знаходять один одного за допомогою DNS-запитів. У цьому випадку кожен сервіс реєструється в DNS-системі з унікальним ім'ям, і клієнтські сервіси можуть звертатися до інших сервісів, використовуючи ці імена. DNS-система автоматично розв'язує ці імена в актуальні IP-адреси інстансів сервісів. Цей підхід широко використовується в Kubernetes (через CoreDNS) та Docker Swarm.

Як працює **DNS-based discovery**:

1. Сервіс реєструє себе в DNS-системі при запуску, надаючи унікальне ім'я (наприклад, `user-service.default.svc.cluster.local` у Kubernetes).
2. Клієнтський сервіс звертається до іншого сервісу, використовуючи його DNS-ім'я.
3. DNS-система розв'язує це ім'я в актуальні IP-адреси інстансів сервісу.
4. Клієнт встановлює з'єднання з одним з отриманих IP-адрес і виконує запит.
5. Якщо інстанс недоступний, клієнт може повторити DNS-запит, щоб отримати оновлений список IP-адрес.

```
RestTemplate template = new RestTemplate();  
String url = "http://user-service.default.svc.cluster.local"; // DNS-ім'я
```

Kubernetes (через CoreDNS) автоматично знайде IP.

Переваги та недоліки **DNS-based discovery**:

Переваги:

- Широке використання та підтримка в багатьох системах (Kubernetes, Docker Swarm);
- Простота використання, оскільки DNS є стандартним механізмом у мережах;
- Автоматичне оновлення IP-адрес через TTL записів у DNS.

Недоліки:

- Затримки через DNS-запити, особливо якщо TTL високий і записи не оновлюються швидко;
- Обмежена гнучкість у виборі алгоритмів балансування навантаження (залежить від можливостей DNS-сервера);
- Можливі проблеми з кешуванням DNS на стороні клієнта, що може призводити до використання застарілих IP-адрес.

Key-Value Store-based discovery (виявлення на основі сховища ключ-значення) — це підхід до Service Discovery, при якому сервіси реєструють свої інстанси в розподіленому сховищі ключ-значення (наприклад, Consul, Etcd, Zookeeper). Клієнтські сервіси звертаються до цього сховища, щоб отримати інформацію про доступні інстанси інших сервісів. Цей підхід забезпечує високу доступність і консистентність даних про сервіси.

Як працює **Key-Value Store-based discovery**:

1. Сервіс реєструє себе в сховищі ключ-значення при запуску, створюючи запис з унікальним ключем (наприклад, `services/user-service/instance-id`) і значенням, що містить інформацію про розташування (IP-адресу, порт, метадані).
2. Клієнтський сервіс звертається до сховища ключ-значення, щоб отримати список доступних інстансів потрібного сервісу.
3. Клієнт отримує список інстансів і використовує алгоритм балансування навантаження (наприклад, `round-robin`, `random`) для вибору одного з них.
4. Клієнт встановлює пряме з'єднання з обраним інстансом і виконує запит.
5. Якщо обраний інстанс недоступний, клієнт може повторити запит до сховища ключ-значення, щоб отримати оновлений список інстансів і спробувати знову.

```
RestTemplate template = new RestTemplate();
String url = "http://user-service"; // логічне ім'я
```

Consul API або інший клієнт автоматично знайде IP.

Переваги та недоліки **Key-Value Store-based discovery**:

Переваги:

- Висока доступність і консистентність даних про сервіси завдяки розподіленій природі сховищ ключ-значення;
- Гнучкість у виборі алгоритмів балансування навантаження на стороні клієнта;
- Підтримка складних сценаріїв, таких як health checks і метадані сервісів.

Недоліки:

- Додаткова складність у налаштуванні та управлінні сховищем ключ-значення;
- Залежність від надійності та продуктивності сховища ключ-значення;
- Потрібно оновлювати всі клієнти при зміні логіки взаємодії зі сховищем.

Критерій	Client-side	Server-side	DNS-based	KV Store-based
Складність на клієнті	Висока	Низька	Низька	Висока
Потрібна підтримка проксі	Ні	Так	Ні	Ні
Балансування трафіку	Гнучке	Централізоване	Базове (round-robin)	Вручну або зовнішнє
Підтримка мов / стеків	Обмежена	Будь-яка	Будь-яка	Через SDK/API
Залежність від бібліотек	Є	Відсутня	Відсутня	Є (наприклад, gRPC+etcd)

Контроль з боку DevOps	Складно	Повний контроль	Обмежено	Повний контроль
Продуктивність	Висока	Нижча через проксі	Висока	Висока (без проксі)

Service Discovery відповідає двом з 2 властивостей [CAP-теорії](#):

1. Бути доступними (A), щоб клієнти могли завжди знайти потрібний сервіс.
2. Бути узгодженими ©, щоб уникнути фальшивих або застарілих записів.

Але при мережевому розділенні (P), доводиться обирати:

- **Consul/Etcd/Zookeeper** → **CP системи**: пріоритет консистентності. Краще зупинити запис, ніж допустити помилкову інформацію.
- **DNS-based discovery** → **AP системи**: завжди дає відповідь, але може бути застарілою (через TTL, кешування).

Де використовується **Service Discovery**:

- **Docker Swarm**: має вбудований механізм Service Discovery, що дозволяє сервісам знаходити один одного за іменами;
- **Kubernetes**: використовує вбудовану DNS-систему (CoreDNS) для автоматичного розв'язання імен сервісів;
- **Consul, Etcd, Zookeeper**: зовнішні системи Service Discovery, які можуть бути інтегровані з Docker, Kubernetes або іншими платформами;
- **AWS Cloud Map, Google Cloud Service Directory**: хмарні сервіси для управління реєстрацією та виявленням сервісів у масштабованих інфраструктурах.

Що дає **Service Discovery**:

- **Автоматизація**: зменшує потребу в ручному налаштуванні мережеских адрес;
- **Балансування навантаження**: деякі системи Service Discovery можуть автоматично розподіляти трафік між кількома інстансами сервісу;
- **Failover**: забезпечує автоматичне перенаправлення трафіку у випадку відмови одного з інстансів сервісу;
- **Горизонтальне масштабування**: дозволяє легко додавати або видаляти інстанси сервісу без необхідності змінювати конфігурацію клієнтів.

Приклади інструментів **Service Discovery**:

- **Docker Swarm**: має вбудований механізм Service Discovery на основі DNS;
- **Kubernetes DNS (CoreDNS)**: вбудована DNS-система для автоматичного розв'язання імен сервісів у кластері;
- **Consul**: потужний інструмент для Service Discovery, health-check-ів та централізованого зберігання конфігурацій;

- **etcd**: розподілене key-value сховище, що використовується Kubernetes для зберігання конфігурації кластера, включно з даними для Service Discovery;
- **Zookeeper**: використовується для координації розподілених систем і як механізм Service Discovery (наприклад, у Kafka, Hadoop);
- **Eureka**: сервісний реєстр, який широко використовується в екосистемі Spring Cloud.

Головні компоненти **Service Discovery**:

- **Реєстратор сервісів (Service Registrar)**: компонент, який відповідає за реєстрацію нових інстансів сервісу в реєстрі та їх видалення у разі зупинки або збою;
- **Сервіс (Service)**: застосунок або мікросервіс, який реєструється в системі Service Discovery;
- **Клієнт Service Discovery**: компонент (або бібліотека), який дозволяє сервісам знаходити інші сервіси за їх логічними іменами;
- **Health Checks**: механізми перевірки стану інстансів сервісів, які гарантують, що клієнти взаємодіють лише з працездатними інстансами;
- **Інтерфейс доступу (DNS або API)**: спосіб, через який клієнти отримують інформацію про доступні сервіси (наприклад, через DNS-імена або REST/gRPC API).

□ 4. Service Discovery vs API Gateway vs Service Mesh

Компонент Основна функція Service Discovery Знаходить інстанси сервісів API Gateway Виступає єдиною точкою входу для клієнтів Service Mesh Контролює трафік між сервісами на рівні L7 (часто в тандемі з discovery)

□□ Поясни, чому Service Mesh (наприклад, Istio, Linkerd) не замінює, а доповнює Service Discovery.

□

□ 5. Failure Scenarios & Resilience • Що буде, якщо зникне весь кластер Consul? • Як Kubernetes відновлює DNS CoreDNS pod? • Як будувати circuit breakers, retries, timeouts при використанні Service Discovery?

□

□ 6. Роль DNS у Service Discovery • Чим DNS-базований discovery відрізняється від API? • Що таке SRV-записи? Як працює dnsRoundRobin? • TTL-залежна кешованість: потенційна проблема в stale-записах

□

□ 7. Безпека Discovery • Хто може реєструвати сервіси? • Може зловмисник зареєструвати фейковий auth-service? • Як захистити доступ до Registry? (mTLS, ACL, JWT)

□

□ 8. Інструменти з досвідом використання

□ Senior має хоча б базовий продакшн-досвід з одним із:

Інструмент Що треба знати Consul ACL, DNS, health checks, sync modes Eureka Self-preservation mode, client cache Kubernetes CoreDNS, headless services, SRV-записи Zookeeper ZNode, ephemeral nodes, leader election

□

□ 9. Класичні питання з інтерв'ю • Поясни різницю між client-side і server-side discovery. • Як ти реалізуєш zero-downtime deployment, якщо IP змінюється? • Що буде, якщо один з інстансів сервісу не працює, але ще зареєстрований? • Чим Kubernetes service discovery відрізняється від Eureka/Consul?

□

□ 10. Тестове завдання або whiteboard:

“Розроби систему, яка динамічно додає і видаляє інстанси сервісу при автоскейлінгу. Клієнти не повинні знати IP-адреси. Як будеш це реалізовувати?”

□

Хочеш — можу скласти для тебе повноцінний “Service Discovery Interview Pack”: • 20+ питань • порівняльні таблиці • cheat sheet • mini-проекти для демонстрації знань (наприклад, Eureka + 2 мікросервіси)

Готовий зробити прямо зараз.

2. DevOps

2.1. Docker Image

WHAT IS A DOCKER IMAGE?



Docker Image — це незмінний (immutable) шаблон, який містить все необхідне для запуску додатку в контейнері: вихідний код, залежності, бібліотеки, системні утиліти та конфігурацію середовища. Образ формується на основі [Dockerfile](#) — спеціального файлу інструкцій, який описує процес створення образу.

Структура Docker Image:

- **Layers** — шари, які являють собою окремі зміни в образі. Кожен шар є результатом виконання команди в Dockerfile;
- **Filesystem** — файловий простір, який містить всі файли та каталоги, необхідні для роботи додатку;
- **Metadata** — метадані, які містять інформацію про образ, такі як автор, версія, команди запуску тощо.

Кожен Docker Image складається з одного або декількох шарів. Шари дозволяють ефективно використовувати дисковий простір, оскільки спільні шари між образами не дублюються. Коли ви змінюєте образ, створюється новий шар, який містить лише зміни, а попередні шари залишаються незмінними.

Layer (слой) — це один з шарів, з яких складається Docker-образ. Кожен слой являє собою результат виконання однієї інструкції (RUN, COPY тощо) в Dockerfile. Docker кешує ці шари, якщо в Dockerfile нічого не змінилось ні інструкція, ні файли які впливають на слой, Docker використовує кеш що прискорює збірку та дозволяє перевикористовувати вже побудовані частини. Але якщо один слой змінився, то всі наступні шари перезбираються

Існують різні типи Docker Image:

- **Base Images** — базові образи, які містять лише операційну систему або мінімальний набір інструментів. Наприклад, `alpine`, `ubuntu`, `debian`.
- **Application Images** — образи, які містять готові до запуску додатки. Наприклад, `nginx`, `mysql`, `node`.
- **Custom Images** — користувацькі образи, створені на основі базових образів з додаванням власних файлів та налаштувань.

Секрети, паролі або ключі не варто зберігати всередині образу. Краще передавати їх через зовнішні змінні середовища, Docker secrets (Swarm), Kubernetes secrets або сторонні сховища (Vault), щоб мінімізувати ризик витоку.

Нижче приведені базові команди для роботи в Docker Image:

Образи створюються за допомогою команди `docker build`, яка читає `Dockerfile` і створює образ на основі вказаних інструкцій. Наприклад:

```
docker build -t myapp:latest .
```

Ця команда створює образ з тегом `myapp:latest` на основі `Dockerfile` в поточній директорії. Це простий варіант створення. Є більш екзотичний як Multi-stage build. **Multi-stage build** - це спосіб розділити сборку на кілька "етапів" (FROM ... AS builder, FROM ... AS runtime) та передавати артефакти, (наприклад, скомпільовані бінарники) з одного етапу в інший. Це зменшує кінцевий образ, оскільки в ньому не залишаються зайві інструменти збірки. Далі наведемо приклад `Dockerfile`:

```
FROM golang:1.16 AS builder
WORKDIR /app
COPY . .
RUN go build -o /app/app

FROM alpine:3.14
COPY --from=builder /app/app /app/app
CMD ["/app/app"]
```

Образи можна завантажувати з Docker Hub або інших реєстрів за допомогою команди `docker pull`. Наприклад:

```
docker pull nginx:latest
```

Ця команда завантажує останню версію образу Nginx з Docker Hub.

Щоб опублікувати свій образ в Docker Hub або іншому реєстрі, використовуйте команду `docker push`. Перед цим потрібно увійти в реєстр за допомогою `docker login`. Наприклад:

```
docker login
```

```
docker push myusername/myapp:latest
```

Образи можна тегувати для зручності ідентифікації. Тегування дозволяє створювати різні версії одного образу. Наприклад:

```
docker tag myapp:latest myapp:v1.0
```

Ця команда створює тег **v1.0** для образу **myapp:latest**. Тепер ви можете використовувати обидва теги для запуску контейнерів.

Щоб переглянути інформацію про образ, використовуйте команду **docker inspect**. Наприклад:

```
docker inspect myapp:latest
```

Ця команда виведе детальну інформацію про образ, включаючи шари, метадані та налаштування.

Для видалення образу використовуйте команду **docker rmi**. Наприклад:

```
docker rmi myapp:latest
```

Ця команда видаляє образ **myapp:latest**. Якщо образ використовується в контейнерах, потрібно спочатку зупинити та видалити ці контейнери.

2.2. Dockerfile

WHAT IS A DOCKERFILE?



FROM
COPY
RUN
CMD

dockerfile — це текстовий файл, який містить набір інструкцій для створення Docker-образу. Він визначає, як збирати образ, які команди виконувати, які файли копіювати, які порти відкривати тощо. Dockerfile використовується командою `docker build` для створення нового образу на основі вказаних інструкцій.

Основні головні інструкції в Dockerfile:

- FROM
- RUN
- CMD
- ENTRYPOINT
- COPY
- ADD
- WORKDIR
- EXPOSE
- ENV
- ARG
- USER
- VOLUME
- LABEL
- HEALTHCHECK

- SHELL

FROM

FROM — це перша й обов'язкова інструкція у більшості Dockerfile. Вона визначає базовий образ, на основі якого буде створено новий Docker-образ. Головні особливості:

- Вказує базовий образ, з якого починається збірка нового образу;
- Може використовуватися декілька разів для multi-stage builds;
- Підтримує різні формати, такі як **FROM <image>[:<tag>]** або **FROM <image>@<digest>** для більшої стабільності збірки;
- Підтримує вказівку архітектури та тегу, наприклад **FROM --platform=linux/amd64 ubuntu:20.04**;
- Підтримує вказівку **AS <name>** для іменування етапу в multi-stage builds.

Головне призначення **FROM** — це визначення базового образу, на якому буде побудований новий образ. В multi-stage builds **FROM** може використовуватися декілька разів для створення різних етапів збірки:

```
FROM ubuntu AS builder
RUN apt-get update && apt-get install -y build-essential
COPY . /app
WORKDIR /app
RUN make

FROM alpine AS runtime
COPY --from=builder /app/myapp /usr/local/bin/myapp
CMD ["/usr/local/bin/myapp"]
```

У цьому прикладі перший **FROM** створює етап збірки, де встановлюються залежності та компілюється додаток. Другий **FROM** створює етап виконання, де копіюється скомпільований додаток з попереднього етапу.

Якщо **FROM** не вказано, Docker не зможе створити образ, оскільки не буде знати, з якого базового образу починати збірку. Це призведе до помилки (Dockerfile parse error: No FROM instruction found) під час виконання команди **docker build**.

Є особливий випадок, коли **FROM** не вказується, це використання **scratch** як базового образу. **scratch** — це порожній образ, який використовується для створення мінімальних образів без операційної системи. Наприклад:

```
FROM scratch
COPY myapp /myapp
CMD ["/myapp"]
```

scratch використовується:

- Коли потрібно створити дуже легкий образ без операційної системи;
- Коли додаток не потребує жодних бібліотек або залежностей, окрім самого виконуваного файлу;
- Для створення образів на основі статично скомпільованих додатків, таких як Go або Rust.

Особливостями `scratch` є:

- Він не містить жодних файлів або директорій, тому всі команди, такі як `RUN`, `COPY`, `ADD`, будуть створювати нові файли в порожньому образі;
- Не підтримує жодних операцій з файловою системою, оскільки це порожній образ;
- Не має shell або інших утиліт, тому не можна виконувати команди, які потребують shell (наприклад, `RUN ls` не спрацює).

В `FROM` можна вказати архітектуру, на якій буде створено образ. Це особливо корисно для multi-stage builds, де різні етапи можуть мати різні архітектури. Наприклад:

```
FROM --platform=linux/arm64 ubuntu:20.04 AS builder
RUN apt-get update && apt-get install -y build-essential
COPY . /app
WORKDIR /app
RUN make
CMD ["/app/myapp"]
```

Оптимальними базовими образами для `FROM` є:

- `alpine` — легкий образ (~5MB) з базовою операційною системою, який містить лише необхідні утиліти;
- `ubuntu` — повноцінний образ з Ubuntu, який містить всі стандартні утиліти та бібліотеки, розмір ~30-60MB;
- `debian/slim` — повноцінний образ з Debian, який також містить всі стандартні утиліти та бібліотеки, розмір ~20MB;
- `scratch` — порожній образ, який використовується для створення мінімальних образів без операційної системи, розмір ~0MB.

Best Practices для `FROM`:

- Використовуй легкі базові образи, такі як `alpine` або `scratch`, якщо це можливо;
- Уникай використання важких базових образів, таких як `ubuntu` або `debian`, якщо не потрібні всі їхні утиліти;
- Використовуй `FROM` з тегами, щоб уникнути проблем з несумісністю версій (наприклад, `FROM ubuntu:20.04` замість `FROM ubuntu`);
- Уникай використання `latest` тегів, оскільки це може призвести до непередбачуваних змін у збірці;
- Використовуй `FROM` з `AS <name>` для іменування етапів у multi-stage builds, щоб зробити

Dockerfile більш читабельним;

- Уникай використання **FROM scratch**, якщо не впевнений, що додаток не потребує жодних бібліотек або утиліт;
- Використовуй **FROM** з **--platform** для вказівки архітектури, якщо потрібно створити образ для різних платформ (наприклад, **FROM --platform=linux/amd64 ubuntu:20.04**).

RUN

RUN — виконує команду на етапі збірки образу. Результат її виконання зберігається як шар (layer) в Docker-образі. Для кожної інструкції **RUN** створюється новий шар, який містить зміни, внесені цією командою. Це дозволяє Docker використовувати кеш для оптимізації збірки образів. Чим більше шарів тим більше образ, тому краще об'єднувати команди в один **RUN**, використовуючи **&&** для зменшення кількості шарів. Наприклад:

```
RUN apt-get update
RUN apt-get install -y curl
RUN rm -rf /var/lib/apt/lists/*
```

Для кожної інструкції **RUN** створюється новий шар, що може призвести до збільшення розміру образу. Краще об'єднати ці команди в один **RUN**, щоб зменшити кількість шарів:

```
RUN apt-get update && \
    apt-get install -y curl && \
    rm -rf /var/lib/apt/lists/*
```

Зараз Docker створить лише один шар, який міститиме всі зміни, внесені цими командами. Це зменшує розмір образу та прискорює збірку. Docker кешує результати кожної інструкції **RUN**, якщо команда не змінилася, Docker використовує кеш для цього шару. Якщо команда не змінилась але ми хочемо, щоб Docker знову виконав її, можна використовувати **--no-cache** при збірці образу:

```
docker build --no-cache -t myimage .
```

Також для мінімізації розміру образу треба використовувати **apt-get clean && rm -rf /var/lib/apt/lists/*** після встановлення пакетів, щоб видалити тимчасові файли, які не потрібні в кінцевому образі. Наприклад:

```
RUN apt-get update && \
    apt-get install -y curl && \
    apt-get clean && \
    rm -rf /var/lib/apt/lists/*
```

Також гарною практикою є створення каталогу з певними правами доступу, щоб уникнути проблем з правами доступу до файлів в контейнері. Наприклад:

```
RUN mkdir -p /app && \
  chown -R username:usergroup /app
```

Що створює каталог `/app` з правами доступу для користувача `username` та групи `usergroup`. Це дозволяє уникнути проблем з правами доступу до файлів в контейнері, якщо додаток працює від імені цього користувача.

Best Practices для `RUN`:

- Використовуй `&&` для об'єднання команд в один `RUN`, щоб зменшити кількість шарів;
- Не додавай секрети або конфіденційну інформацію в `RUN`, оскільки вони залишаться в історії образу;
- Використовуй `--no-cache` при збірці, якщо потрібно примусово виконати команду `RUN`, навіть якщо вона не змінилася;
- Використовуй `apt-get clean && rm -rf /var/lib/apt/lists/*` після встановлення пакетів, щоб зменшити розмір образу;
- Створюй каталоги з певними правами доступу, щоб уникнути проблем з правами доступу до файлів в контейнері.

CMD

CMD — це інструкція Dockerfile, яка визначає команду за замовчуванням, яку Docker виконає при запуску контейнера, якщо не вказано інше під час запуску.

```
CMD ["executable", "param1", "param2"]
```

Ця команда не буде виконана під час збірки образу, а лише при запуску контейнера (`docker run`). Якщо вказано декілька аргументів, вони будуть передані як список. Може бути тільки одна інструкція `CMD` в Dockerfile, якщо їх декілька, то буде використана остання. Така інструкція не потребує `ENTRYPOINT`. Не запускається через shell, тому не потрібно використовувати `sh -c` або `bash -c`. Якщо потрібно виконати команду через shell, використовуйте `ENTRYPOINT` з `CMD` для аргументів, або наступний формат (але це застарілий варіант):

```
CMD executable param1 param2
```

Наприклад:

```
CMD nginx -g "daemon off;"
```

Що еквівалентно до:

```
CMD ["/bin/sh", "-c", "nginx -g 'daemon off;']
```

Головне призначення **CMD** — це визначення команди, яка буде виконана при запуску контейнера. Інструкція **CMD** у **Dockerfile** використовується **Docker**’ом під час запуску контейнера, а не під час збірки образу. Вона задає команду за замовчуванням, яка буде виконана, якщо користувач не вкаже свою. Наприклад є **Dockerfile**:

```
FROM alpine
CMD ["echo", "Hello, World!"]
```

І при запуску користувач вказує свою команду, наприклад "ls -la":

```
docker run myimage echo "ls -la"
```

У цьому випадку команда **CMD** буде ігноруватися, і буде виконана команда користувача. Якщо ж користувач не вказує свою команду, то буде виконана команда з **CMD**, тобто **echo "Hello, World!"**. Якщо **CMD** використовується разом з **ENTRYPOINT**, то **CMD** буде передаватися як аргументи до **ENTRYPOINT**. Наприклад:

```
FROM alpine
ENTRYPOINT ["ping"]
CMD ["google.com"]
```

```
docker run myimage
# → ping google.com

docker run myimage yahoo.com
# → ping yahoo.com
```

Також для того, щоб перевизначити **CMD** у **docker-compose** файлі, можна використовувати **command**:

```
version: '3'

services:
  myservice:
    image: myimage
    command: ["echo", "Hello from docker-compose!"]
```

Якщо не вказано **command**, то буде виконана команда з **CMD** в **Dockerfile**.

У **Kubernetes**, **CMD** може бути перевизначено в **Deployment** або **Pod** через поле **command**. Наприклад:

```
apiVersion: apps/v1
kind: Deployment
```

```

metadata:
  name: my-deployment
spec:
  replicas: 1
  selector:
    matchLabels:
      app: myapp
  template:
    metadata:
      labels:
        app: myapp
    spec:
      containers:
        - name: mycontainer
          image: myimage
          command: ["ping", "localhost"] # Перезаписує ENTRYPOINT/CMD, що
еквівалентно ENTRYPOINT ["echo", "Hello from Kubernetes!"]
          args: ["google.com"]          # Аргументи для команди, що еквівалентно CMD
["google.com"]

```

Головне про CMD:

- Використовується для визначення команди за замовчуванням, яка буде виконана при запуску контейнера;
- Якщо не вказано, то буде виконана команда з **CMD**;
- Якщо користувач вказує свою команду при запуску, то команда з **CMD** буде ігноруватися;
- Якщо використовується разом з **ENTRYPOINT**, то **CMD** буде передаватися як аргументи до **ENTRYPOINT**;
- Якщо в Dockerfile є декілька **CMD**, то буде використана остання команда;
- **CMD** не виконується якщо вказана інша команда при запуску контейнера або якщо використовується **--entrypoint**, тоді **CMD** буде ігноруватися (якщо не використовується як аргумент).

ENTRYPOINT

ENTRYPOINT — визначає основну команду, яку Docker завжди виконає, коли запускається контейнер. На відміну від **CMD**, вона не перезаписується аргументами docker run, якщо явно не використати **--entrypoint**. Головною метою **ENTRYPOINT** є запусити процес який має стартувати разом з контейнером. Приймати параметри від **CMD** або аргументів командного рядка (**docker run**).

Головною метою **ENTRYPOINT** є запуск процесу, який має стартувати разом з контейнером в незалежності від того, чи вказані додаткові аргументи при запуску контейнера. Це дозволяє створити контейнер, який завжди виконує певну задачу, наприклад, запуск веб-сервера або бази даних. Далі наведемо приклади використання **ENTRYPOINT**, першим прикладом є простий запуск Nginx сервера(наприклад):

```
FROM nginx
```

```
ENTRYPOINT ["nginx", "-g", "daemon off;"]
```

Також **ENTRYPOINT** може використовуватись для запуску та передачі аргументів в shell-скрипт, наприклад:

```
FROM alpine
COPY start.sh /usr/local/bin/start.sh
ENTRYPOINT ["/usr/local/bin/start.sh"]
```

```
#!/bin/sh
echo "Запуск з аргументами: $@"
exec "$@"
```

При запуску контейнера з таким **ENTRYPOINT**, ви можете передати додаткові аргументи, які будуть доступні в скрипті **start.sh** через **\$@**. Наприклад:

```
docker run myimage echo "Hello, World!"
# → Запуск з аргументами: Hello, World!
$Hello, World!
```

Головне про **ENTRYPOINT**:

- Використовується для визначення основної команди, яка завжди виконується при запуску контейнера;
- Не перезаписується аргументами **docker run**, якщо не використовується **--entrypoint**;
- Дозволяє створити контейнер, який завжди виконує певну задачу;
- Може використовуватись разом з **CMD** для передачі аргументів до основної команди;
- Використовується для запуску процесів, які мають стартувати разом з контейнером;
- Може бути використано для запуску shell-скриптів, які приймають аргументи;
- Якщо в Dockerfile є декілька **ENTRYPOINT**, то буде використана остання команда;
- **ENTRYPOINT** може бути використано для запуску процесів, які мають стартувати разом з контейнером, наприклад, веб-серверів або баз даних;
- Якщо використовується разом з **CMD**, то **CMD** буде передаватися як аргументи до **ENTRYPOINT**;
- Якщо в Dockerfile є декілька **ENTRYPOINT**, то буде використана остання команда;
- **ENTRYPOINT** не виконується якщо використовується **--entrypoint**.

CMD vs ENTRYPOINT

Особливість	CMD	ENTRYPOINT
Призначення	Команда за замовчуванням	Основна команда (завжди виконується)

Перевизначення	Легко перевизначається через <code>docker run</code>	Може вимагати <code>--entrypoint</code>
Взаємодія	Може передавати аргументи до ENTRYPOINT	Команду не можна легко змінити

COPY

COPY використовується в Dockerfile для копіювання файлів і директорій з локальної файлової системи (контексту збірки) в файлову систему образу. Головні особливості:

- Копіює файли/директорії з контексту збірки в контейнер;
- `--chown` дозволяє встановити власника та групу для скопійованих файлів;
- Не розпаковує архіви (.tar.gz, .zip тощо);
- Не підтримує URL, тобто не завантажує файли з Інтернету.

ADD

ADD — інструкція Dockerfile, яка копіює файли з локального контексту збірки або з віддалених URL у файлову систему контейнера, але з додатковими можливостями, яких не має **COPY**. Головними особливостями **ADD** є:

- Копіює файли/директорії з контексту збірки в контейнер;
- Підтримує розпакування архівів (наприклад, .tar.gz) при копіюванні;
- Підтримує завантаження файлів з віддалених URL;
- Має параметр `--chown` для встановлення власника та групи для скопійованих файлів.
- Не розпаковує архіви з URL, тобто якщо ви вказуєте URL, то архів не буде розпакований.

Деколько прикладів, якщо необхідно розпакувати архів:

```
ADD myapp.tar.gz /usr/src/myapp/
```

То результатом виконання буде:

```
/usr/src/myapp/file1.txt
/usr/src/myapp/file2.txt
...
```

Але якщо ви вказуєте URL:

```
ADD https://example.com/myapp.tar.gz /usr/src/myapp/
```

то результатом буде:


```
/usr/src/myapp/myapp.tar.gz
```

Тобто якщо архів вказаний в команді **ADD** розташований локально то він буде розпакований, але якщо він вказаний як URL, то він буде просто скопійований в контейнер без розпакування.

COPY vs ADD Відмінність між COPY та ADD:

- **COPY** копіює файли/директорії з контексту збірки в контейнер.
- **ADD** робить те ж саме, але також може автоматично розпаковувати архіви та завантажувати файли по URL. Рекомендується частіше використовувати **COPY** (більш передбачувано), **ADD** — тільки коли потрібні додаткові можливості.

Як результат можна зробити висновок, що:

- **COPY** — це простий і передбачуваний спосіб копіювання файлів з контексту збірки в контейнер;
- **ADD** — це більш потужний інструмент, який дозволяє розпаковувати архіви та завантажувати файли з URL, але може бути менш передбачуваним;
- Рекомендується використовувати **COPY** для простих копіювань, а **ADD** — тільки коли потрібні додаткові можливості, такі як розпакування архівів або завантаження файлів з URL.

WORKDIR

WORKDIR встановлює робочу директорію всередині контейнера для наступних інструкцій (**RUN**, **CMD**, **ENTRYPOINT** і т.д.). Якщо каталогу немає, він буде створений.

EXPOSE

EXPOSE вказує, що контейнер слухає порт вказаний в цій команді. Це не впливає на роботу контейнера всередині Docker, але це корисно для читабельності та може враховуватися інструментами оркестрації.

ENV

ENV встановлює змінні середовища для контейнера, які будуть доступні на етапі виконання контейнера (runtime).

Ці змінні:

- можуть використовуватись як під час збірки Dockerfile, так і в запущеному контейнері, оскільки зберігаються в образі;
- дозволяють задавати конфігураційні параметри, які можуть бути змінені при запуску контейнера;
- завжди присутні в контейнері, що робить його більш передбачуваним і “персистентним”;
- можуть бути використані в командах, додатках або скриптах, що працюють всередині контейнера.

Для перевизначення змінних середовища при запуску можна використати:

- `docker run -e KEY=VALUE`
- `docker run --env KEY=VALUE`

Також їх можна заздалегідь визначити в Dockerfile через:

```
ENV APP_HOME=/usr/src/app
WORKDIR $APP_HOME
RUN echo "App home is set to $APP_HOME"
```

Головне призначення **ENV** — це встановлення змінних середовища, які будуть доступні на етапі виконання контейнера. Це дозволяє задавати конфігураційні параметри, які можуть бути змінені при запуску контейнера, і робить його більш передбачуваним і “персистентним”. Гнучкість для користувача: якщо ви хочете дозволити користувачам змінювати параметри роботи контейнера на етапі запуску, використовуйте **ENV**. Ці значення можна легко перевизначити за допомогою флага **-e** або через `docker-compose`.

ARG

Для того щоб передати аргументи сборки в Dockerfile, використовуйте інструкцію **ARG**. При зборці передавайте **--build-arg**. Наприклад Dockerfile:

```
ARG APP_VERSION=latest
RUN echo "Version: $VERSION"
```

Сборка:

```
docker build --build-arg APP_VERSION=1.2 .
```

Головне призначення **ARG** дозволяє задавати змінні на етапі зборки образу (build-time). Ці змінні використовуються тільки під час зборки і не доступні коли образ зібран. Тобто ці змінні використовуються тільки в Dockerfile і не зберігаються в контейнері та недоступні під час виконання контейнера. Використання **ARG** є вірним вибором для зберігання сенсатів даних (токени, ключі API тощо) для того щоб вони не зберігалися в кінцевому образі. Але краще уникати використання таких даних в будь-якому вигляді, тому що навіть у випадку з **ARG**, ці дані можуть залишитися в історії збірки Docker-образу (наприклад, в проміжних шарах).

ARG vs ENV

Параметр	ARG	ENV
Використання	На етапі збірки образу	На етапі виконання контейнера
Доступність	Доступны только на этапе сборки	Доступны на этапе выполнения контейнера

Перевизначення	Можно переопределить с помощью <code>--build-arg</code> при сборке	Можно переопределить с помощью <code>-e</code> при запуске контейнера
Сохранение в образе	Не сохраняется в образе	Сохраняется в образе
Цель	Используется для передачи временных значений при сборке	Используется для задания окружения приложений внутри контейнера

ARG/ENV best practices:

- Використовуйте **ARG** для змінних, які потрібні тільки на етапі збірки. Це хороший спосіб уникнути витоків даних, які не потрібні на етапі виконання.
- Використовуйте **ENV** для змінних, які потрібні на етапі виконання контейнера. Це дозволяє гнучко налаштовувати контейнер, передаючи параметри додатку та середовища.
- Мінімізуйте кількість змінних середовища **ENV**, що містять чутливі дані. Хоча **ENV** може бути зручним для передачі конфігурацій, це не найкращий спосіб для зберігання секретів, оскільки вони зберігаються в образі і можуть бути витягнуті. Краще використовувати секрети Docker (наприклад, через Docker Swarm або Kubernetes), якщо потрібно працювати з чутливими даними.
- Чітко розмежуйте етапи збірки та виконання: Розуміння того, на якому етапі (збірки чи виконання) потрібні змінні, допоможе правильно вибрати між **ARG** і **ENV**.
- Використовуйте спільне використання **ARG** і **ENV**, коли потрібно передати значення на етапі збірки, а потім зберегти його для використання в контейнері.

ARG/ENV Summary:

- ARG** — це змінні, які використовуються тільки на етапі збірки Dockerfile. Вони не зберігаються в кінцевому образі і не доступні під час виконання контейнера.
- ENV** — це змінні середовища, які доступні на етапі виконання контейнера. Вони зберігаються в образі і можуть бути використані додатками всередині контейнера.

USER

USER — визначає користувача, від імені якого будуть виконуватись усі наступні інструкції (**RUN**, **CMD**, **ENTRYPOINT**, тощо), а також процес у контейнері під час запуску. Це дуже важлива інструкція для безпеки, тому що за замовченням все запускається від root користувача. Це зручно для встановлення пакетів але небезпечно для запуску додатків. Простий приклад:

```
FROM alpine

# Під root встановлюємо залежності
RUN apt-get update && apt-get install -y curl && rm -rf /var/lib/apt/lists/*

# Створюємо користувача
RUN adduser -D myuser
```

```
# Переключаємось  
USER myuser  
  
# Далі всі команди – без root-доступу  
CMD ["echo", "Hello from myuser!"]
```

Коли використовуєш **USER** треба переконатись що користувач має права доступу до потрібних дерікторій:

```
FROM alpine  
  
WORKDIR /app  
RUN adduser -D myuser  
USER myuser  
RUN --chown=myuser:myuser /app
```

VOLUME

VOLUME — створює точку монтування для зберігання даних, які не зберігаються в образі. Це дозволяє зберігати дані, які можуть змінюватись під час виконання контейнера, і робить їх доступними навіть після перезапуску контейнера. Наприклад:

```
VOLUME ["/data", "/logs"]
```

При цьому Docker створює нові томи для **/data** та **/logs**, монтує їх в вказану директорию в контейнері, дані в цих директоріях не зберігаються у шарах образу, а зберігаються в том. І при перезапуску контейнеру дані зберігаються. Але неможна задати ім'я тома, тому що Docker сам створює тома з випадковими іменами. Але за необхідністю можна створити **іменованій тома** вручну і підключити його до контейнера.

LABEL

LABEL — використовується для додавання метаданих (інформації) до Docker-образу у вигляді пар ключ=значення. Ці дані не впливають на виконання контейнера, але корисні для ідентифікації, автоматизації та документування. Docker рекомендує OCI Image Specification для ключів:

```
LABEL org.opencontainers.image.title="My App" \  
      org.opencontainers.image.description="Awesome app" \  
      org.opencontainers.image.version="1.0.0" \  
      org.opencontainers.image.licenses="MIT" \  
      org.opencontainers.image.source="https://github.com/user/repo"
```

Для того щоб переглянути метадані образу, можна використати команду:

```
docker inspect myimage --format='{{json .Config.Labels}}'
```

HEALTHCHECK

HEALTHCHECK — це інструкція, яка дозволяє перевіряти “здоров’я” контейнера під час його роботи. Docker автоматично виконує команду перевірки з певним інтервалом і встановлює статус:

- `healthy` — контейнер працює коректно
- `unhealthy` — контейнер не відповідає
- `starting` — контейнер ще запускається

Це дозволяє автоматично виявляти проблеми з контейнером і вживати заходів, наприклад, перезапустити його. Приклад використання:

```
FROM nginx:alpine

HEALTHCHECK --interval=30s --timeout=3s --retries=3 \
  CMD curl -f http://localhost/ || exit 1
```

Для перевірки статусу здоров’я контейнера можна використати команду:

```
docker inspect --format='{{json .State.Health}}' mycontainer
```

або

```
docker ps
```

І як результат можна побачити статус здоров’я контейнера в колонці **STATUS**.

```
Up 30 seconds (healthy)
Up 1 minute (unhealthy)
Up 10 seconds (starting)
```

Важливі нюанси:

- **HEALTHCHECK** не перезапускає контейнер автоматично (але можна налаштувати Docker Swarm/Kubernetes для рестарту при `unhealthy`);
- Впливає на CI/CD (деякі оркестратори чекають статусу `healthy` перед деплоєм);
- Використовуй легкі команди (перевірка має бути максимально швидкою (кілька секунд))
- Винось складну логіку в скрипт, наприклад:

```
COPY healthcheck.sh /usr/local/bin/
```

SHELL

SHELL — визначає, яку командну оболонку (shell) Docker буде використовувати для виконання інструкцій на кшталт **RUN**, **CMD** та **ENTRYPOINT** (коли вони задаються у shell-форматі). Тобто потрібен для того щоб змінити інтерпретатор команд, який буде використовуватись для виконання команд в Dockerfile. За замовчуванням це **/bin/sh -c**, але можна змінити на іншу оболонку, наприклад, **/bin/bash -c** або **/bin/zsh -c**. Приклад:

```
SHELL ["/bin/bash", "-c"]
RUN echo "Hello from bash" \
```

Для того, щоб забезпечити якість Dockerfile ніж пушити до репозиторію, можна використовувати наступні підходи:

1. **Сборка образу.** Запустіть **docker build .** для зборки образу. Перевірте, що образ збирається без помилок.
2. **Запуск контейнера.** Запустіть контейнер зі збудованим образом через **docker run**. Перевірте, що контейнер запускається і працює коректно.
3. **Юніт-тести.** Напишіть юніт-тести для Dockerfile. Наприклад, використовуючи Molecule або Testcontainers.

Кращі практики при роботі в dockerfile:

- Базовый образ — бери минимальный (alpine, slim, scratch), фиксируй версию, не используй latest;
- Меньше слоев — объединяй команды RUN, удаляй кешы и временные файлы;
- .dockerignore — исключай ненужные файлы (.git, node_modules, логи);
- Не root — создавай пользователя и запускай от него (USER app);
- ENTRYPOINT + CMD — разделяй основную команду и аргументы по умолчанию;
- Multi-stage build — собирай в одном образе, запускай в другом (минимальном);
- Безопасность — только нужные пакеты, без секретов в Dockerfile, регулярные обновления;
- Використовуйте **COPY** замість **ADD**, якщо не потрібно автоматичне розпакування архівів або завантаження з URL;
- Використовуйте **ARG** для передачі змінних збірки, щоб зробити Dockerfile більш гнучким.

2.3. Docker Volume

WHAT IS A DOCKER VOLUME?



Docker Volumes — це механізм для зберігання даних поза життєвим циклом контейнера тобто по за файловою системою контейнера. Контейнер можна перестворити, а дані в томі залишаться. А також як варіант ділитися файлами між хостом та контейнером або між контейнерами. Але при одночасному підключенню одного тому до декількох контейнерів можуть виникнути конфлікти, тому потрібно переконатися, що додатки підтримують спільний доступ до томів.

Розрізняють три види томів:

- **Anonymous Volume** — це том, який створюється без імені і використовується тільки в одному контейнері;
- **Bind Mount** — це прив'язка до каталогу або файлу на хості, який буде доступний в контейнері;
- **Named Volume** — це том, який створюється з іменем і може бути використаний в декількох контейнерах.

Anonymous Volumes — це томи, які створюються автоматично без імені при використанні VOLUME в [Dockerfile](#) або `docker run -v /path`. Очевидним мінусом є те, що їх важко ідентифікувати та керувати ними та використовувати повторно.

Bind Mount — це механізм, який дозволяє прив'язати каталог або файл на хості до контейнера. Це дозволяє контейнеру отримувати доступ до даних на хості або зберігати дані на хості. Приклад:

```
docker run -v /host/path:/container/path ...
```


Мінус цього варіанту в, тому що є залежність від файлової системи.

Для того, щоб створити тома з іменами треба використовувати `docker-compose` або `docker run -v`:

```
# Створюємо том з іменем вручну
docker volume create my_named_volume

# Запускаємо контейнер з підключеним іменованим томом
docker run -d \
  --name my-container \
  -v my_named_volume:/app/data \
  my-image

# або
docker run -d \
  --name my-container \
  --mount type=volume,source=my_named_volume,target=/app/data \
  myimage
```

або в `docker-compose`

```
version: '3.8'
services:
  my-service:
    image: my-image
    volumes:
      - my_named_volume:/app/data
```

Може бути зручно для повторного використання даних між контейнерами або для резервного копіювання.

Різниця між **Bind Mount** та **Anonymous Volume**:

- **Bind Mount** напряму зв'язує каталог/файл хоста з контейнером;
- **Anonymous Volume** зберігається в управляемому Docker (наприклад, `/var/lib/docker/volumes`), не прив'язаний напряму до шляху на хості.

Різниця між **Named Volume** та **Bind Mount**:

- **Named Volume** — це окремий об'єкт в Docker, який можна використовувати в інших контейнерах;
- **Bind Mount** — це просто шлях на хості, який можна використовувати тільки в одному контейнері.

Різниця у швидкості доступу до даних: **Named Volume*** швидше, але **Bind Mount** має більше можливостей.

Порівняльна таблиця:

Тип тома	Доступ до даних	Повторне використання	Залежність від хоста	Розшарювати між контейнерами	Головне використання
Anonymous Volume	Через Docker	Ні	Ні	Технічно так, але незручно	Тимчасові дані
Named Volume	Через Docker	Так	Ні	Так	Постійне зберігання даних
Bind Mount	Через хост	Ні	Так	Так, якщо вказати той самий шлях	Розробка, тестування

Зазвичай логи контейнера виводяться в stdout/stderr (для **docker logs**) або підключаються до централізованої системи логування. Якщо потрібно зберігати логи на хості, можна підключити volume або bind mount для каталогу логів.

Використання томів в Docker дозволяє:

- Зберігати дані поза контейнером, що дозволяє зберегти їх при перезапуску або видаленні контейнера;
- Ділитися даними між контейнерами;
- Зберігати конфігураційні файли, логи та інші дані, які можуть змінюватися під час роботи контейнера;
- Використовувати дані з хоста в контейнері, наприклад, для розробки або тестування.

Драйвера доступні в volume:

Драйвер	Тип зберігання	Призначення
local	Локальна ФС (/var/lib/docker/volumes)	За замовчуванням, швидке локальне зберігання
nfs	NFS-сервер	Спільний доступ між кількома хостами
tmpfs	Оперативна пам'ять	Дуже швидке, але тимчасове зберігання
aws	Amazon EBS	Блочне сховище AWS
azurefile	Azure File Storage	Хмарне сховище Microsoft Azure
gcs	Google Cloud Storage	Хмарне сховище Google
efs	AWS Elastic File System	Розподілене хмарне сховище AWS

rexray	Багато платформне блочне сховище	AWS, GCP, Azure, OpenStack
flocker	Кластерне зберігання	Для розподілених систем
portworx	Розподілене блокове сховище	Для Kubernetes/Docker кластерів
ceph	CephFS або RBD	Масштабоване кластерне сховище

Для того щоб створити том з використанням специфічного драйвера, можна використовувати команду:

```
docker volume create --driver <ім'я_драйвера> [--opt ключ=значення] <ім'я_тому>
```

Далі наведемо приклади створення томів, першим спробуємо створити з використанням драйвера **tmpfs**:

```
docker volume create \
  --driver local \
  --opt type=tmpfs \
  --opt device=tmpfs \
  --opt o=size=200m \
  tmpfs_volume
```

Розберемо команду більш детально:

- **docker volume create** — команда для створення нового тому;
- **--driver local** — вказує, що ми використовуємо локальний драйвер;
- **--opt type=tmpfs** — вказує тип тома як **tmpfs**, що означає, що дані будуть зберігатися в оперативній пам'яті;
- **--opt device=tmpfs** — вказує, що ми використовуємо **tmpfs** як пристрій для зберігання даних;
- **--opt o=size=200m** — вказує, що розмір тома обмежений 200 мегабайтами;
- **tmpfs_volume** — ім'я створюваного тому.

Ще одним прикладом буде створення тому з використання NFS-драйвера:

```
docker volume create \
  --driver local \
  --opt type=nfs \
  --opt o=addr=192.168.1.100,rw,nfsvers=4,tcp \
  --opt device=:/nfs_data \
  my_nfs_volume
```

Розберемо команду більш детально:

- `docker volume create` — команда для створення нового тому;
- `--driver local` — вказує, що ми використовуємо локальний драйвер;
- `--opt type=nfs` — вказує тип тома як `nfs`, що означає, що дані будуть зберігатися на NFS-сервері;
- `--opt o=addr=192.168.1.100,rw` — вказує адресу NFS-сервера та режим доступу (читання і запис);
- `--opt device=:/nfs_data` — вказує шлях до каталогу на NFS-сервері, який буде використовуватися як том;
- `my_nfs_volume` — ім'я створюваного тому.

Основні опції для `docker volume create --driver local --opt:`

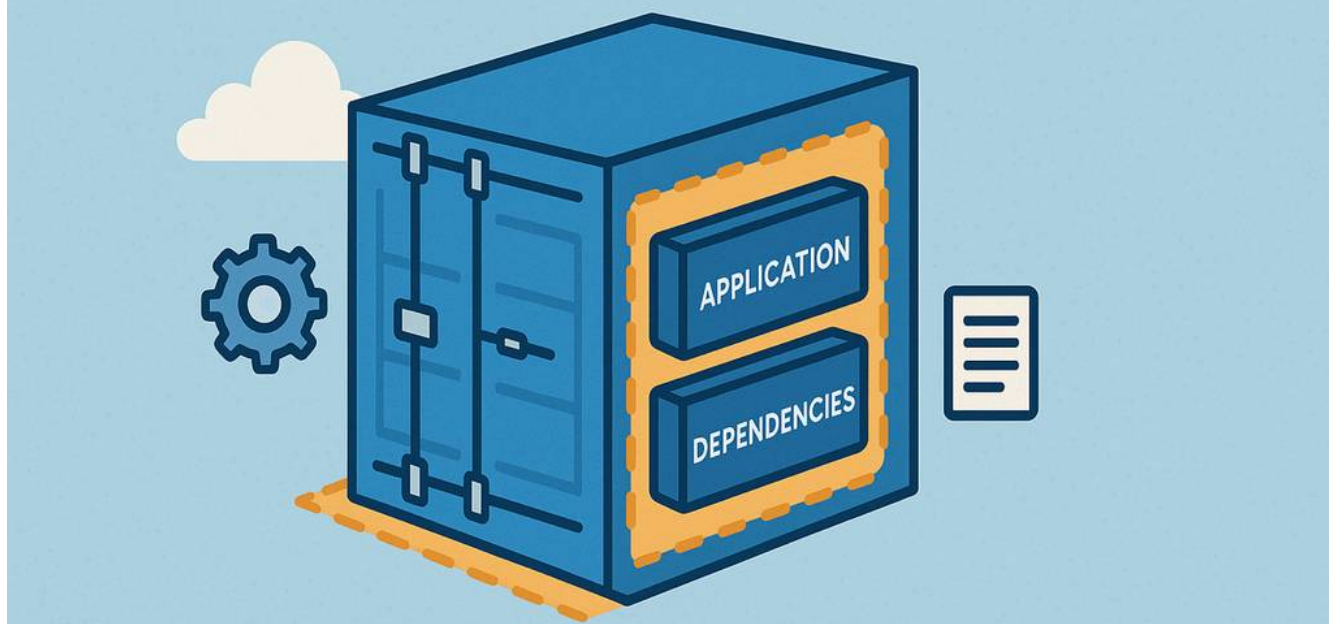
Опція	Приклад Значення	Опис
type	tmpfs, nfs, ceph, aws, azurefile	Тип тома, який буде створено.
device	/path/to/device, :/nfs_data	Шлях до пристрою або каталогу, який буде використовуватися як том.
o	rw, ro, size=200m, addr=192.168.1.100	Опції монтування, як у mount -o
uid	1000	Власник файлів (User ID)
gid	1000	Група файлів (Group ID)
mode	0700	Права доступу до каталогу
nfsvers	3 або 4	Версія NFS
noatime	—	Не оновлювати час доступу до файлів (оптимізація швидкості)
soft / hard	—	Поведінка при розриві зв'язку з NFS (soft — повертає помилку, hard — чекає відновлення)
timeo	600	Таймаут у десятих частках секунди для NFS
tcp / udp	—	Протокол передачі для NFS
vers	4.1	Версія протоколу NFS
nolock	—	Вимкнення блокування файлів в NFS

Команди для роботи з томами в Docker

- `docker volume ls` - список всіх томів;
- `docker volume inspect <volume_name>` - отримання детальної інформації про том `<volume_name>`, включаючи його шлях на хості;
- `docker volume rm <volume_name>` - видалення тому `<volume_name>`;
- `docker volume rm $(docker volume ls -q)` - видалення всіх томів (будьте обережні, це видалить всі томи без підтвердження);
- `docker volume prune` - видалення всіх не використовуваних томів;
- `docker volume prune --force` - примусове видалення всіх не використовуваних томів без підтвердження;
- `docker run -v <volume_name>:/path/in/container <image>` - запуск контейнера з підключеним томом `<volume_name>` до шляху `/path/in/container`;
- `docker run --mount type=volume,source=<volume_name>,target=/path/in/container <image>` - запуск контейнера з підключеним томом `<volume_name>` до шляху `/path/in/container` (альтернативний синтаксис);
- `docker volume create <volume_name>` - створення нового тому з іменем `<volume_name>`;
- `docker volume create --name <volume_name>` - створення нового тому з іменем `<volume_name>`;
- `docker volume create --driver <driver_name> <volume_name>` - створення нового тому з використанням специфічного драйвера `<driver_name>`;
- `docker volume create --opt <key>=<value> <volume_name>` - створення нового тому з додатковими опціями `<key>=<value>`.

2.4. Docker Container

WHAT IS A DOCKER CONTAINER?



Docker Container — це ізольований userspace-процес або звичайний linux процес запусканий з використанням kernel features для ізоляції та обмеження ресурсів, яке дозволяє запускати додатки та сервіси в контейнеризованому вигляді. Працює поверх ядра Linux (або через WSL2 на Windows) та створюється на основі Docker-образа з використанням [containerd](#) як high-level контейнерного runtime і [runc](#) як low-level OCI-сумісного runtime для запуску процесів контейнера. Контейнери використовують спільне ядро операційної системи, але мають власні файлові системи, мережеві інтерфейси та процеси. Це дозволяє запускати додатки в ізоляції від інших процесів на хості, забезпечуючи легкість, швидкість та портативність. Контейнер реалізує ізоляцію процесів, файлової системи, мережі, користувачів та ресурсів через kernel-примітиви: [namespaces](#), [cgroups](#), [seccomp](#), [AppArmor/SELinux](#), [capabilities](#).

Контейнери не зберігають стан (stateless by design), що робить їх ідеальними для мікросервісної архітектури, де кожен сервіс може бути запусканий в окремому контейнері. Вони також легко масштабуються та інтегруються в CI/CD процеси. Persistence даних може бути досягнута через [volumes](#), bind mounts або external services (наприклад, бази даних). Дуже добре масштабується в кластерних середовищах, таких як [Kubernetes](#), [Swarm](#) або Nomad.

Docker Container vs Віртуальна машина

Віртуальна машина — це полноценная ОС, яка віртуалізує апаратне забезпечення та запускає окрему гостьову ОС. **Контейнер** — це ізольоване середовище, яке використовує ядро хостової ОС та запускає окремі процеси. Контейнери запускаються швидше, оскільки вони не потребують емуляції апаратного середовища та завантаження окремої ОС.

Table 1. Порівняння з віртуальними машинами:

Характеристика	Контейнер Docker	Віртуальна машина (VM)
Ядро ОС	Спільне з хостом	Власне, повноцінне ядро
Швидкість запуску	Секунди	Хвилини
Ресурси	Мінімальні	Значно більше
Вага (типово)	10–100 МБ	Гігабайти
Ізоляція	Через kernel features (LXC, namespaces , cgroups)	Через гіпервізор
Портативність	Висока	Середня
Мета	Запуск окремих процесів	Запуск повноцінної ОС

Table 2. Продуктивність

Параметр	Контейнер	Віртуальна машина
Startup time	~50–500 мс	~10–60 сек
CPU overhead	мінімальний	помітний
Memory overhead	мінімальний	високе споживання
I/O performance	залежить від storage driver	близьке до native (з pass-through)



Storage drivers: OverlayFS (default), aufs (deprecated), devicemapper, ZFS, Btrfs — кожен має нюанси по performance, stability та layered caching.

Архітектура запуску контейнера:

1. [Docker CLI](#)
2. [Docker Engine \(dockerd\)](#)
3. [containerd \(high-level runtime\)](#)
4. [runc \(OCI low-level runtime\)](#)
5. [Linux kernel \(namespaces + cgroups\)](#)



Контейнери не віртуалізують апаратне забезпечення — замість цього вони опираються на ядро хоста і [lightweight isolation primitives](#).

Контейнер складається з:

- [Файлова система](#)
- [Процеси](#)
- [Мережа](#)
- [Ресурси](#)
- [network](#)

Файлова система контейнера Docker побудована на основі шарів Docker-образа, які використовують **каскадне об'єднане монтування** (OverlayFS, UnionFS) з **copy-on-write**. Це дозволяє створювати ізольовані файлові системи для кожного контейнера, де зміни записуються лише у верхній gw-шар, не змінюючи базовий образ. Це дозволяє економити місце та швидко створювати нові контейнери.

Процеси в контейнері запускаються в ізольованому PID namespace, що дозволяє їм не бачити процеси на хості та інших контейнерах. Це забезпечує безпеку та ізоляцію, оскільки процеси в контейнері не можуть взаємодіяти з процесами на хості або в інших контейнерах.

Контейнери обмежуються в використанні ресурсів через control groups (**cgroups**). Це дозволяє контролювати використання CPU, пам'яті, I/O та інших ресурсів, що забезпечує стабільність та безпеку системи. Наприклад, можна обмежити кількість CPU, пам'яті або I/O, які може використовувати контейнер.

Контейнер має власний network namespace, що дозволяє йому мати окрему IP-адресу, DNS та порти. Це забезпечує ізоляцію мережі між контейнерами та хостом, дозволяючи кожному контейнеру мати свій власний мережевий стек. Контейнери можуть спілкуватися між собою через **Docker network**. Але можуть виникнути проблеми спілкування між контейнерами, якщо вони не налаштовані правильно або використовують різні мережеві драйвери. Для пошуку проблем з мережею між контейнерами можна використовувати:

- **docker exec -it <container> sh** для доступу до контейнера і виконання команд ping, curl або netcat до інших контейнерів.
- **docker network inspect <network>** для перевірки налаштувань мережі.

Контейнери забезпечують безпеку через кілька механізмів:

- **PID namespace** — ізолює процеси, що дозволяє контейнеру не бачити процеси на хості.
- **NET namespace** — забезпечує окремий мережевий стек, що дозволяє контейнеру мати власні IP-адреси, порти та DNS.
- **MNT namespace** — ізолює файлову систему, що дозволяє контейнеру мати власний простір монтування (/ , /proc , /sys).
- **UTS namespace** — ізолює hostname, що дозволяє контейнеру мати власний hostname.
- **IPC namespace** — забезпечує власну пам'ять для inter-process communication, що дозволяє контейнеру мати власні IPC-ресурси.
- **User namespace** — дозволяє UID 0 у контейнері не бути root на хості, що забезпечує додатковий рівень безпеки.

Для того, щоб запускати контейнери під non-root користувачем, використовуйте флаг **--user**, наприклад:

```
docker run --user 1000 myimage
```

це є бест практикою для безпеки. Тому що при взломі якщо зломисник отримає доступ до

контейнера, він не отримає root-права на хості. Це мінімізує можливий збиток при взломі. `--user 1000` означає, що контейнер буде запущений від користувача з UID 1000 (На більшості Linux систем це перший створений користувач). Як що в контейнері нема користувача с таким UID, то процеси все одно запустяться і можуть не мати доступу до файлів або дерікторій. Це може бути корисно для запуску додатків, які не потребують root-доступу, або для обмеження доступу до системних ресурсів.

Додаткові механізми безпеки:

- **Seccomp** — фільтрація системних викликів, що дозволяє обмежити доступ до небезпечних системних викликів.
- **AppArmor/SELinux** — мандатне управління доступом, що дозволяє контролювати доступ до ресурсів на основі політик безпеки.

Namespaces — это механизм ядра Linux, который позволяет изолировать системные ресурсы между группами процессов.

Namespace	Изолирует	Детали
pid	ID процессов	Процессы не видят друг друга
net	Сетевые интерфейсы и IP-стек	Своя сеть, DNS, маршруты
mnt	Файловую систему и монтирования	Изоляция /, /proc, /sys
ipc	Shared memory	Не пересекаются IPC-ресурсы
uts	Hostname, domainname	У контейнера свой hostname
user	UID/GID	Разделение root внутри контейнера и на хосте
cgroup	Контроль над ресурсами	Контейнер — член своего cgroup

Головними аспектами безпеки Docker-контейнерів є:

- Не запускайте контейнери під root-користувачем всередині контейнера.
- Відключайте зайві Capabilities.
- Використовуйте AppArmor/SELinux профілі.
- Скануйте образи на уразливості.
- Своєчасно оновлюйте.

Переваги та недоліки контейнерів Docker

Перевагами контейнерів є:

- Однакове середовище в розробці, тестуванні й продакшені (проблема "It works on my machine" вирішена)
- Швидкий запуск і масштабування

- Економія ресурсів у порівнянні з віртуальними машинами
- Портативність — запуск на будь-якій платформі з Docker Engine
- Легко інтегруються у CI/CD процеси

Недоліками є:

- Безпека — ізоляція не така сильна, як у віртуальних машинах
- Складність управління — потребують додаткових інструментів для оркестрації (Kubernetes, Docker Swarm)
- Залежність від хостової ОС — не можуть запускати різні ОС (наприклад, Windows-контейнер на Linux)

Далі перейдемо до практичного використання Docker-контейнерів.

```
docker run [OPTIONS] IMAGE [COMMAND] [ARG...]
```

Флагі яки використовуються при запуску контейнера:

- `--name` — вказує ім'я контейнера;
- `-d` або `--detach` — запускає контейнер у фоновому режимі;
- `-p` або `--publish` — публікує порти контейнера на хості;
- `-v` або `--volume` — монтує том або bind mount;
- `-e` або `--env` — передає змінні середовища в контейнер;
- `--env-file` — передає змінні середовища з файлу;
- `--cpus` — обмежує використання CPU (наприклад, `--cpus="1.5"`);
- `--cpu-period` — встановлює період для обмеження CPU (наприклад, `--cpu-period=100000`);
- `--cpu-quota` — встановлює квоту для обмеження CPU (наприклад, `--cpu-quota=50000`);
- `-m` або `--memory` — обмежує використання пам'яті;
- `--memory-swap` — обмежує використання пам'яті з урахуванням swap;
- `--cpuset-cpus` — обмежує використання певних CPU;
- `--pids-limit` — обмежує кількість процесів у контейнері (наприклад, `--pids-limit=100`);
- `--device-read-bps` — обмежує швидкість читання з пристрою (наприклад, `--device-read-bps /dev/sda:1mb`);
- `it` — запускає контейнер в інтерактивному режимі (tty) (поєднання `-i` та `-t`);
- `-i` — інтерактивний режим (stdin залишається відкритим);
- `-t` — виділяє псевдотермінал (tty);
- `--rm` — автоматично видаляє контейнер після завершення;
- `--restart` — вказує політику перезапуску контейнера (наприклад, `always`, `unless-stopped`, `on-failure`);

- `--network` — вказує мережу, до якої підключити контейнер;
- `--entrypoint` — вказує точку входу для контейнера (перезаписує значення з образу);
- `--mount` — більш гнучкий спосіб підключення томів (наприклад, `--mount source=mydata,target=/app/data`).
- `--privileged` — надає контейнеру всі можливості хостової системи (не рекомендується використовувати без потреби).

Для того, щоб опублікувати порт контейнера на зовні, використовуйте флаг `-p` або `--publish`. Наприклад:

```
docker run -p <HOST_PORT>:<CONTAINER_PORT> myimage
```

де:

- `<HOST_PORT>` — порт на хості, на якому буде доступний контейнер;
- `<CONTAINER_PORT>` — порт всередині контейнера, на якому працює додаток.

Для передачі аргументів контейнеру використовуйте флаг `-e` або `--env <file>`, наприклад:

```
docker run -e APP_ENV=prod -e DB_PASS=secret myimage
```

або якщо змінних забагато, то можна передати їх з файлу за допомогою флага `--env-file`:

```
docker run --env-file <file> myimage
```

`--cpu` — це ручна огортка над `--cpu-period` та `--cpu-quota`, яка дозволяє легко обмежити використання CPU контейнером. Але якщо треба точне управління, наприклад, змінити довжину періодів (для real-time додатків), задати квоту не в цілих ядрах або адаптувати поведінку в embedded-середовищах, то `--cpu-period` + `--cpu-quota` дають повний контроль. Наведемо приклад:

```
docker run --rm -it \
  --cpu-period=100000 \
  --cpu-quota=50000 \
  ubuntu
```

у цьому випадку контейнер буде обмежений до $50000 / 100000 = 0.5 \rightarrow 50\%$ одного ядра CPU. Це означає, що контейнер може використовувати максимум 50% CPU в кожному періоді в 100000 мікросекунд (або 0.1 секунди) тобто 0.05 секунди при цьому 0.05 секунди не працює.

Для запуску контейнера в інтерактивному режимі з доступом до командного рядка, використовуйте флаг `-it` (поєднання `-i` та `-t`). Це дозволяє вам взаємодіяти з контейнером через термінал. Наприклад:

```
docker exec -it <CONTAINER_NAME> bash
```

Тобто ця команда дозволяє виконувати команди в середині контейнера. В цьому ми викликаємо оболонку `bash`. Також це може бути корисним якщо на просто треба перевірити наявність файлу в контейнері, наприклад:

```
docker exec -it <CONTAINER_NAME> ls -la
```

Але в цілому це дозволяє виконувати будь які команди в середині контейнеру.

Для того, щоб автоматично видалити контейнер після завершення його роботи, використовуйте флаг `--rm`. Це дозволяє уникнути накопичення зупинених контейнерів. Наприклад:

```
docker run --rm myimage
```

А якщо взяти попередній приклад, наприклад ви створили Docker Image і просто хочете субу перевірити чи є там наприклад якийсь файл то команда

```
`docker run --rm <IMAGE_NAME> ls -l <PATH_TO_FILE>`
```

Тобто докер запускає контейнер виконує команда і по завершенні команди видаляє контейнер.

Для того щоб запустити контейнер без мережевого стека, використовуйте:

```
docker run --network=none myimage
```

Флаг `--privileged` дає контейнеру майже необмежений доступ до хостової системи. Це може бути корисно для деяких сценаріїв, але також значно знижує безпеку, тому його слід використовувати з обережністю. Використання цього флага:

- Усі Linux Capabilities (capabilities) надаються контейнеру;
- Доступ до всіх пристроїв в `/dev`;
- Можливість монтувати файлові системи, змінювати мережеві інтерфейси та інші системні ресурси;
- Робота з AppArmor/SELinux може бути ослаблена.

```
docker run --privileged myimage
```

Контейнер може бути в наступних станах:

Running

Контейнер має стан Running, коли він запущений і виконує процеси. У цьому стані всі процеси активні, і контейнер може обробляти запити.

Paused

Контейнер має стан Paused, коли він призупинений. У цьому стані всі процеси зупинені, але зберігають свій стан. Це може бути корисно для тимчасового призупинення роботи контейнера без його зупинки. Для того щоб призупинити контейнер, використовуйте команду:

```
docker pause <container_id>
```

Команда **docker pause** надсилає сигнал SIGSTOP усім процесам контейнера. Всі процеси **заморожуються** (їх стан зберігається в пам'яті, але вони не виконуються). І контейнер переходить в стан Paused. Для відновлення контейнера до стану Running, використовуйте команду:

```
docker unpause <container_id>
```

Ця команда може бути корисною для тимчасового призупинення роботи контейнера, наприклад, для проведення технічного обслуговування або безпечного оновлення мережевого або дискового шару (у деяких сценаріях CI/CD або live-debug). Або для зупинки енергоємного процесу.

Stopped

```
docker stop <container_id>
```

Команда **docker stop** посилає сигнал SIGTERM процесам в контейнері, даючи їм час для коректного завершення. Якщо процеси не завершуються протягом 10 секунд, Docker відправляє сигнал SIGKILL для примусового завершення. Але якщо необхідно одразу відправити сигнал SIGKILL то використовується команда **docker kill** яка моментально зупиняє контейнер не даючи йому час на коректну зупинку процесів. Контейнер зупиняється, але його дані зберігаються, і ви можете знову запустити його пізніше. І переходить у стан Exited. Для відновлення контейнера до стану Running, використовуйте команду:

```
docker start <container_id>
```

Exited

Контейнер має стан Exited, коли він завершив свою роботу. Це може статися через успішне завершення процесу або через помилку. Контейнер все ще існує, і ви можете переглядати його логи або перезапустити його. Якщо контейнер одразу після запуску переходить в цей стан, це може бути через те, що процес в контейнері завершився з помилкою або не був запущений або був короткостроковим, наприклад **docker run alpine ls -la**. Це може

трапитись через декілька причин:

- Невірна команда **ENTRYPOINT** або **CMD** в Dockerfile;
- Відсутність необхідних залежностей або бібліотек в образі;
- Помилки в коді додатку, які призводять до аварійного завершення;
- Неправильні аргументи або змінні середовища, які передані при запуску контейнера.

Dead

Контейнер має стан Dead, коли він не відповідає на запити. Це може статися через серйозні помилки в процесах контейнера або проблеми з ресурсами. Контейнер все ще існує, але його неможливо перезапустити без видалення.

Для того, щоб прискорити запуск контейнера:

- Використовуйте легкі образи (alpine, distroless).
- Використовуйте шари з вже завантаженими залежностями.
- Відкиньте непотрібні модулі (spring-devtools).
- Механізми типу GraalVM native-image (якщо виправдано).

Розберемо корисні команди які необхідно знати для роботи з Docker Containers

Для того щоб подивитися логи працюючого контейнера, використовуйте:

```
docker logs <container_id>
```

С флагами **-f** (follow) та **--tail** можна "підглядати" в режимі реального часу.

Для того, щоб обмежити доступ до контейнера ззовні:

- Не публікуйте порти на зовні (не вказуйте -p).
- Використовуйте firewall на хості.
- Застосовуйте політики безпеки (docker network та інші).

Для того щоб передати файли з хоста в контейнер (або навпаки) без пересборки образу, використовуйте bind mount (-v /host/path:/container/path), приклад:

```
docker run -v /host/path:/container/path myimage
```

або копіюйте дані командою **docker cp** приклад:

```
docker cp /host/path mycontainer:/container/path
```

Для того, щоб зберегти дані при перезапуску контейнера БД, використовуйте **volume**, наприклад:

```
docker run -v mydb-data:/var/lib/postgresql/data postgres
```

Перезапуск контейнера не затроне дані в томі.

Для того, щоб додати Capabilities до контейнера використовуєте флаг `--cap-add`, наприклад:

```
docker run --cap-add=<cap_name> myimage
```

або для того щоб видаляти Capabilities з контейнера, використовуйте флаг `--cap-drop`:

```
docker run --cap-drop=<cap_name> myimage
```

Для того, щоб перевірити які Capabilities має контейнер, використовуйте команду `docker inspect`:

```
docker inspect <container_id>
```

та перегляньте секцію CapAdd/CapDrop. По замовчуванню є базовий набір (наприклад, NET_RAW та інші).

За замовчуванням JVM може "не бачити" обмеження cgroups. `-XX:MaxRAMPercentage` допомагає JVM коректно визначати доступну пам'ять. Інакше може бути `OutOfMemoryError`, якщо JVM буде думати, що доступна вся пам'ять хоста. `-XX:MaxRAMFraction` працює аналогічно, але відносно обсягу пам'яті. Приклад використання:

```
docker run -e JAVA_OPTS="-XX:MaxRAMPercentage=75.0" <IMAGE_NAME>
```

що означає, що JVM використовує 75% доступної пам'яті.

```
docker run -e JAVA_OPTS="-XX:MaxRAMFraction=0.5" <IMAGE_NAME>
```

що означає, що JVM використовує половину доступної пам'яті.

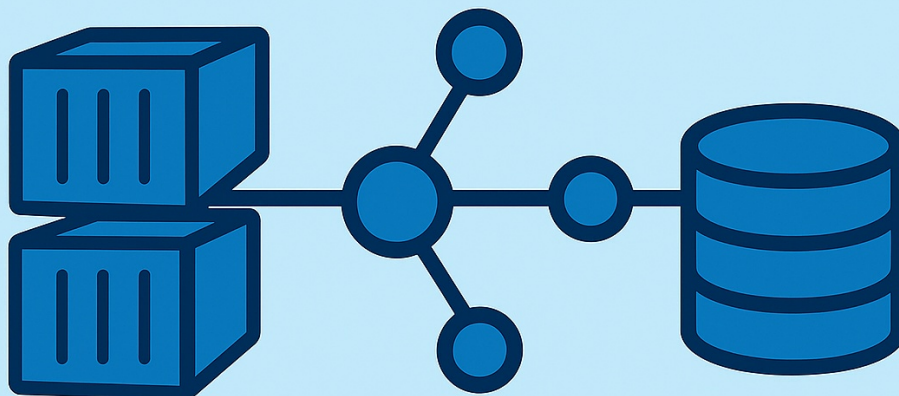
Best practices:

- Мінімізуй образи (multi-stage builds, Alpine, distroless).
- Run as non-root + drop capabilities.
- Використовуй ReadOnly rootfs (`--read-only`), tmpfs для /tmp.
- Monitor: docker stats, cAdvisor, Prometheus exporters.
- Scan images на CVE (Trivy, Gype, Snyk).
- Використовуй image signing (Cosign, Notary v2).

- Автоматичне оновлення — через GitOps або image watchers.

2.5. Docker Network

WHAT ARE DOCKER NETWORKS

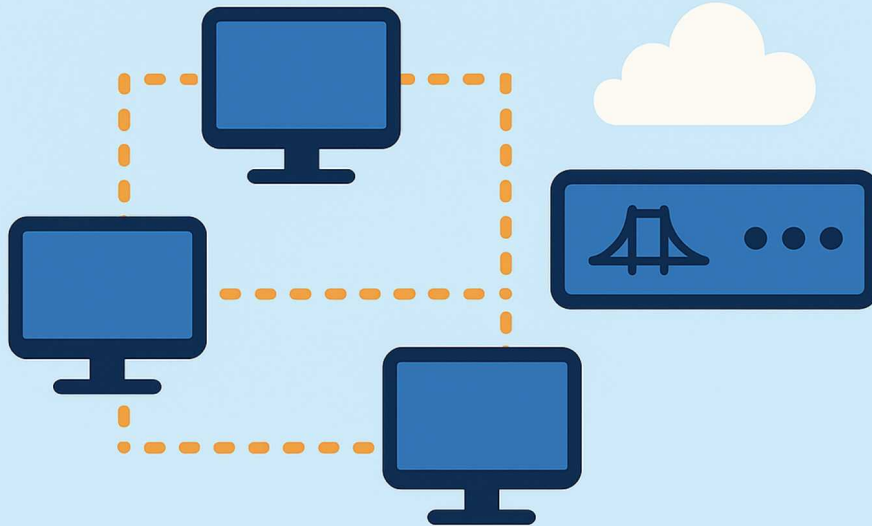


Docker Network — це механізм, який дозволяє контейнерам спілкуватися один з одним та з зовнішнім світом. Docker автоматично створює мережу **bridge**, яка використовується за замовчуванням для всіх контейнерів. Але можна створювати власні мережі для кращої організації та безпеки. Існують кілька типів мереж в Docker:

- **Bridge**;
- **Host**;
- **Overlay**;
- **Macvlan**;
- **None**.

2.5.1. Bridge Network

WHAT IS A DOCKER BRIDGE NETWORK?



Мережа **bridge** є мережею, яка створюється Docker за замовчуванням. Створюється мост (docker0) на хості, контейнери під'єднуються до нього що дозволяє контейнерам на одному хості спілкуватися між собою. Кожен контейнер отримує власну IP-адресу в межах цієї мережі. Це забезпечує ізоляцію між контейнерами, але дозволяє їм спілкуватися один з одним через IP-адреси або імена хостів (вони ж назви контейнерів).

Основні властивості мережі **bridge**:

- Кожен контейнер отримує власну приватну IP-адресу у віртуальній мережі;
- Спілкування між контейнерами в одній мережі bridge відбувається напряму;
- Для доступу ззовні використовуються механізми **NAT** та проброс портів (-p);
- **DNS**-розв'язання для імен контейнерів підтримується всередині user-defined bridge;
- Існує базова мережева ізоляція між контейнерами та хостом, а також між контейнерами з різних bridge-мереж.

Мережа **bridge** використовується в наступних випадках:

- **Ізольоване середовище для сервісів:** Наприклад, backend і база даних в одному docker-compose, які спілкуються між собою;
- **Тестування сервісів:** Простий спосіб підняти кілька взаємопов'язаних контейнерів для локального тестування;
- **Контейнери за замовчуванням:** При створенні контейнера без явно вказаної мережі, Docker підключає його до default bridge network.

Bridge pros and cons:

Pros:

- **Ізоляція:** Контейнери ізолювані від хосту та інших bridge-мереж;
- **Гнучкість:** Можна створювати власні bridge-мережі та керувати [DNS](#), alias-ами та правилами доступу;
- **Масштабованість:** Підходить для роботи з кількома контейнерами на одному хості;
- **Безпечніше, ніж host**, оскільки порти не відкриті на хості за замовчуванням.

Cons:

- **Затримка:** Трохи вища, ніж у режимі host, через використання [NAT](#);
- **Неінтуїтивність для новачків:** Потрібен проброс портів для доступу до сервісів ззовні;
- **Конфігурація [DNS](#) обмежена в default bridge:** Для розширених можливостей треба створювати user-defined bridge network;
- **Обмежене спілкування між bridge-мережами**, якщо не налаштоване вручну (через iptables або зовнішній проху).

Потенційні проблеми з режимом [bridge](#):

- Контейнери не можуть спілкуватися між bridge-мережами без додаткової конфігурації;
- Можуть виникати конфлікти портів при пробросі, якщо декілька контейнерів намагаються використовувати один і той же порт на хості;
- Якщо неправильно налаштовані правила [NAT](#), зовнішній трафік може не доходити до контейнера;
- Проблеми з [DNS](#)-розв'язанням у default bridge, якщо покладатися лише на імена контейнерів.

Головні особливості мережі [bridge](#):

- Тип мережі за замовчуванням для контейнерів у Docker;
- Ідеально підходить для невеликих або середніх проєктів з локальним мережевим зв'язком між контейнерами;
- Забезпечує базову ізоляцію, баланс між безпекою та зручністю;
- Для складніших кейсів — рекомендовано створювати user-defined bridge network, яка підтримує повноцінний [DNS](#), alias-и, контроль доступу.

User-defined bridge network — це мережа, створена користувачем за допомогою команди `docker network create --driver bridge <network-name>`. Вона має більше можливостей, ніж default bridge network, яку Docker створює автоматично.

[User-defined bridge network](#) — це кастомна мережа типу bridge, яка забезпечує:

- **Кращу ізоляцію між контейнерами:** контейнери в різних user-defined bridge мережах не можуть спілкуватися між собою без додаткової конфігурації;
- Автоматичне [DNS](#)-розв'язання імен контейнерів;

- Можливість вказувати aliases (псевдоніми);
- Гнучке управління з'єднаннями між контейнерами.

Відмінності від **default bridge network**:

Функція	default bridge	user-defined bridge
DNS-розв'язання по імені контейнера	Немає	Є
Автоматичне з'єднання	Немає	Можна з'єднати по імені
Псевдоніми (aliases)	Немає	Можна вказувати
Більше контролю	Мінімальний	Повний контроль
Мережева ізоляція між bridge-мережами	Є	Є

Навіщо використовувати **user-defined bridge**:

- Коли потрібно, щоб декілька контейнерів легко “бачили” один одного по імені;
- Для docker-compose, який автоматично створює таку мережу;
- Коли потрібна краща безпека та контроль ізоляції;
- Для налаштування складніших топологій без host або macvlan.

2.5.2. Host Network

WHAT IS A DOCKER HOST NETWORK?



Host — це тип мережі при якому контейнер не має власної IP-адреси, а використовуватиме IP-адресу хоста. Контейнер не матиме власної IP-адреси, а використовуватиме IP-адресу хоста. Це може бути корисно для високопродуктивних застосунків покращуючи продуктивність, але знижує рівень ізоляції контейнерів.

Основні властивості мережі **host**:

- Контейнер не має власної IP-адреси, а використовує IP-адресу хоста;
- Контейнер відкриває порти напряму на хості, без використання **NAT**;
- Контейнер використовує **DNS** хоста, що дозволяє йому спілкуватися з іншими контейнерами та зовнішнім світом без **NAT**;
- Відсутня мережева ізоляція між контейнером та хостом;

Мережа **host** використовується в наступних випадках:

1. Максимальна продуктивність: коли важлива швидкість та низька затримка, наприклад, для високопродуктивних застосунків або сервісів, які потребують швидкого доступу до мережі;
2. Моніторинг та метрик: Наприклад, Prometheus node_exporter або cadvisor — їм потрібно мати доступ до всієї системи;
3. Тестування локальних мереж: коли потрібно тестувати мережеві застосунки на локальному хості без використання **NAT**;

Host pros and cons:

Pros:

- **Відсутність **NAT****, прямий доступ до мережі, немає прокидання портів (-p);
- **Низька затримка (latency)**, оскільки немає додаткового рівня абстракції між контейнером та хостом;
- **Контейнер має повний доступ до мережі хоста**, що дозволяє йому спілкуватися з іншими контейнерами та зовнішнім світом без **NAT**;
- **Простота інтеграції з системами**, які не підтримують Docker networking, оскільки контейнер використовує мережу хоста.

Cons:

- **Втрата ізоляції** між контейнером та хостом, що може призвести до проблем з безпекою;
- **Конфлікти портів**, якщо порт вже зайнятий хостом, контейнер не стартує;
- **Безпека**: зловмисний код в контейнері має повний мережевий доступ до хоста, що може призвести до компрометації системи;
- **Не підтримується на Mac/Windows**: оскільки для Docker Desktop імітує мережу, але режим **host** там не працює. В Linux режим **host** дозволяє контейнеру безпосередньо використовувати мережевий стек хост-машини, що означає, що контейнер бачить ті ж мережеві інтерфейси, IP-адреси та порти, що і хост. На macOS/Windows Docker працює через віртуальну машину, (наприклад, Hyper-V на Windows або Docker Desktop на macOS),

тому контейнери, запущені в цій віртуальній машині, не мають прямого доступу до мережевого стеку хост-машини. Спроба використання режиму **host** на macOS/Windows призведе до того, що контейнер не зможе отримати доступ до мережевих ресурсів хоста або, навпаки, хост не зможе безпосередньо звернутися до ресурсів контейнера. Тому для досягнення бажаної функціональності потрібно використовувати альтернативний підхід, наприклад, використання пробросу портів або налаштування мережі через віртуальну машину;

- **Ускладнення налаштувань firewall/iptables**, оскільки контейнер не відрізняється від інших системних процесів.

Потенційні проблеми з режимом **host**:

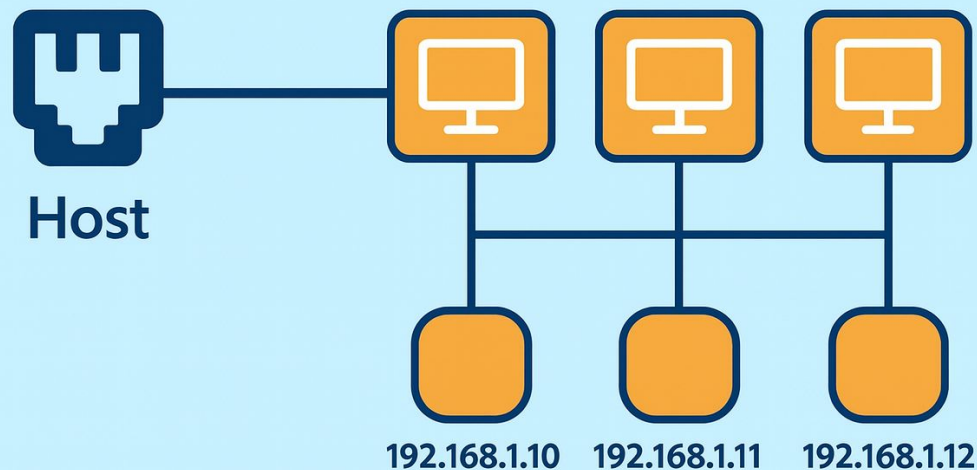
- Конфлікти портів з хост-системою, оскільки контейнери використовують порти хоста без NAT;
- Відсутність мережевої ізоляції, що може призвести до проблем з безпекою;
- Складніше управляти безпекою, оскільки контейнер фактично "бачить" все, що є на хості.

Головні особливості мережі **host**:

- Висока продуктивність, оскільки немає додаткового рівня абстракції між контейнером та хостом;
- Контейнер може отримувати доступ до всіх мережевих інтерфейсів хоста;
- Використовується для високопродуктивних застосунків, де важлива швидкість та низька затримка.

2.5.3. Macvlan Network

WHAT IS A DOCKER MACVLAN NETWORK?



Мережа **Macvlan** — це тип мережі Docker, який дозволяє контейнерам мати власні MAC-адреси та IP-адреси, як фізичні пристрої в локальній мережі. Це дозволяє контейнерам бути безпосередньо доступними в локальній мережі, що може бути корисно для інтеграції з існуючими мережевими інфраструктурами або для застосунків, які потребують унікальних MAC-адрес.

Основні властивості мережі **macvlan**:

- Кожен контейнер отримує власну MAC-адресу та IP-адресу в межах фізичної мережі;
- Контейнери можуть спілкуватися з іншими пристроями в локальній мережі безпосередньо, як фізичні машини;
- Підтримує різні режими роботи (bridge, private, passthru, 802.1q) для гнучкого налаштування мережі;
- Ізоляція між контейнерами та хостом, що підвищує безпеку;
- Не підтримує **DNS**-розв'язання за замовчуванням, потребує додаткової конфігурації для імен контейнерів.

Мережа **macvlan** використовується в наступних випадках:

1. Інтеграція з існуючими мережами: Коли контейнери повинні бути видимими в локальній мережі з унікальними MAC-адресами;
2. Застосунки, які потребують унікальних MAC-адрес: Наприклад, мережеві служби або застосунки, які використовують MAC-адреси для автентифікації чи ліцензування;
3. Високопродуктивні мережеві застосунки: Коли потрібен прямий доступ до мережі без

додаткових рівнів абстракції;

4. Ізольовані мережеві середовища: Коли потрібно забезпечити ізоляцію між контейнерами та хостом для підвищення безпеки.

macvlan pros and cons:

Pros:

- Унікальні MAC-адреси для кожного контейнера, що дозволяє їм бути видимими в локальній мережі;
- Прямий доступ до мережі без додаткових рівнів абстракції, що покращує продуктивність;
- Ізоляція між контейнерами та хостом, що підвищує безпеку;
- Гнучкі режими роботи для різних сценаріїв використання.

Cons:

- Складність налаштування, особливо для новачків у Docker та мережах;
- Потребує підтримки на рівні мережевого обладнання (наприклад, комутатори повинні підтримувати роботу з MAC-адресами);
- Не підтримує **DNS**-розв'язання за замовчуванням, що ускладнює доступ до контейнерів за іменами;
- Може виникати конфлікт MAC-адрес, якщо не налаштовано правильно;
- Обмежена підтримка на деяких платформах (наприклад, Windows, MacOS).

Потенційні проблеми з режимом **macvlan**:

- Конфлікти MAC-адрес, якщо кілька контейнерів або пристроїв у мережі мають однакові MAC-адреси;
- Складність налаштування мережевого обладнання для підтримки MAC-адрес контейнерів;
- Відсутність **DNS**-розв'язання за замовчуванням, що ускладнює доступ до контейнерів за іменами;
- Проблеми з безпекою, якщо контейнери мають прямий доступ до фізичної мережі без належної ізоляції;
- Обмежена підтримка на деяких платформах, що може ускладнити використання в гетерогенних середовищах.

Головні особливості мережі **macvlan**:

- Дозволяє контейнерам мати унікальні MAC-адреси та IP-адреси, як фізичні пристрої в локальній мережі;
- Підтримує різні режими роботи (bridge, private, passthru, 802.1q) для гнучкого налаштування мережі;

- Забезпечує ізоляцію між контейнерами та хостом для підвищення безпеки;
- Ідеально підходить для інтеграції з існуючими мережевими інфраструктурами або для застосунків, які потребують унікальних MAC-адрес.

2.5.4. None Network

WHAT IS A DOCKER NONE NETWORK?



None — це тип мережі, при якому контейнер не має жодного мережевого інтерфейсу. Контейнер ізольований від будь-якої мережі, включаючи інші контейнери на тому ж хості. Це може бути корисно для застосунків, які не потребують мережевого доступу або коли потрібна максимальна ізоляція.

Основні властивості мережі **none**:

- Контейнер не має жодного мережевого інтерфейсу;
- Контейнер ізольований від будь-якої мережі, включаючи інші контейнери на тому ж хості;
- Використовується для застосунків, які не потребують мережевого доступу;
- Забезпечує максимальну ізоляцію між контейнером та хостом, а також між контейнерами.

Мережа **none** використовується в наступних випадках:

1. Застосунки без мережевих потреб: Коли контейнер запускає застосунок, який не потребує мережевого доступу, наприклад, для обробки даних або виконання локальних завдань;

2. Підвищена безпека: Коли потрібно забезпечити максимальну ізоляцію контейнера від мережі для підвищення безпеки;
3. Тестування: Для тестування застосунків у повністю ізольованому середовищі без мережевого впливу;
4. Спеціалізовані сценарії: Коли контейнер використовується для специфічних завдань, які не потребують мережевого доступу, наприклад, для запуску скриптів або обробки файлів.

None pros and cons:

Pros:

- **Максимальна ізоляція:** Контейнер повністю ізольований від мережі, що підвищує безпеку;
- **Відсутність мережевих конфліктів:** Оскільки контейнер не має мережевого інтерфейсу, неможливі конфлікти портів або IP-адрес;
- **Простота використання:** Легко налаштовується для застосунків, які не потребують мережевого доступу;
- **Зменшення поверхні атаки:** Відсутність мережевого інтерфейсу знижує ризик атак через мережу.

Cons:

- **Відсутність мережевого доступу:** Контейнер не може спілкуватися з іншими контейнерами або зовнішніми сервісами;
- **Обмежене використання:** Підходить лише для специфічних сценаріїв, де мережевий доступ не потрібен;
- **Складність інтеграції:** Неможливо інтегрувати контейнер з іншими сервісами або базами даних, які знаходяться поза контейнером;
- **Відсутність можливості моніторингу через мережу:** Неможливо використовувати мережеві інструменти для моніторингу або управління контейнером.

Потенційні проблеми з none:

- Неможливість спілкування з іншими контейнерами або зовнішніми сервісами, що може обмежити функціональність застосунку;
- Відсутність мережевого доступу може ускладнити інтеграцію з іншими системами або сервісами;
- Складність у випадках, коли застосунок раптово потребує мережевого доступу, що вимагатиме зміни конфігурації контейнера;
- Відсутність можливості використовувати мережеві інструменти для моніторингу або управління контейнером, що може ускладнити діагностику проблем.

Головні особливості мережі none:

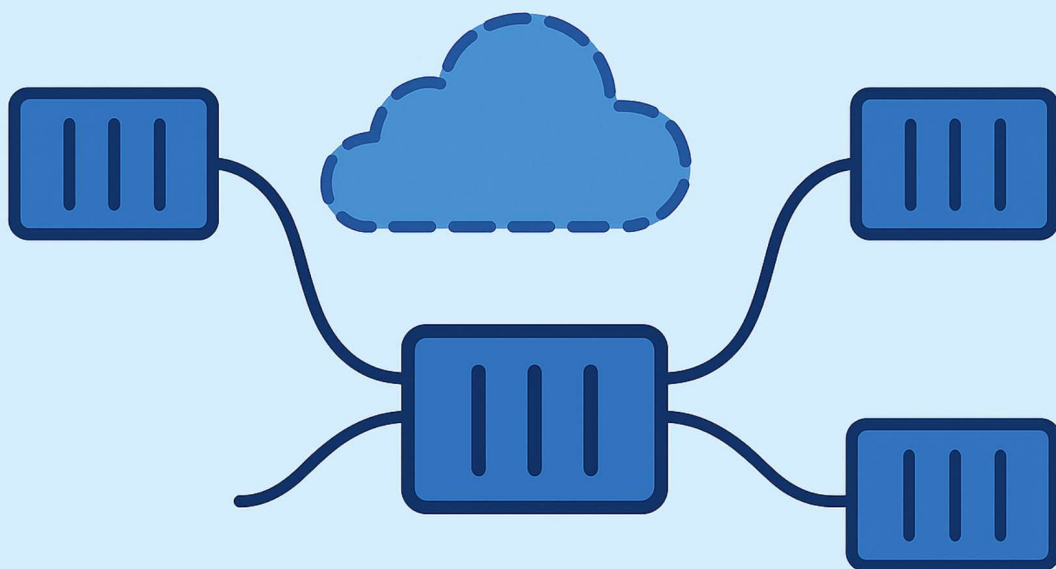
- Контейнер не має жодного мережевого інтерфейсу, що забезпечує максимальну ізоляцію

від мережі;

- Ідеально підходить для застосунків, які не потребують мережевого доступу або коли потрібна максимальна безпека;
- Використовується для специфічних сценаріїв, таких як обробка даних, виконання локальних завдань або тестування у ізольованому середовищі;
- Забезпечує зменшення поверхні атаки за рахунок відсутності мережевого інтерфейсу.

2.5.5. Overlay Network

WHAT IS A DOCKER OVERLAY NETWORK?



overlay — це тип мережі Docker, який дозволяє контейнерам спілкуватися між собою, які запуснені на різних Docker-хостах, об'єднуючи їх у єдину віртуальну мережу. Overlay-мережа працює поверх (over) фізичної мережі і забезпечує спілкування між контейнерами, які знаходяться на різних вузлах кластеру (наприклад, у Docker Swarm або Kubernetes). **Overlay** мережі забезпечують високу ізоляцію та безпеку, дозволяючи контейнерам на різних хостах спілкуватися між собою через [Ingress](#).

Основні властивості мережі **overlay**:

- Дозволяє контейнерам на різних хостах обмінюватися даними як у локальній мережі;
- Використовує VXLAN-тунелювання поверх фізичної мережі;
- Працює лише у кластерному середовищі (Docker Swarm або з зовнішнім key-value store);
- Підтримує [Service Discovery](#) (за іменами сервісів) та балансування навантаження;
- Забезпечує базову ізоляцію між overlay-мережами.

Мережа overlay використовується в наступних випадках:

- Розподілені застосунки: Коли контейнери одного сервісу розгорнуті на різних фізичних/віртуальних машинах;
- Docker Swarm / Kubernetes: Для забезпечення міжвузлового зв'язку між контейнерами;
- [Service Discovery](#) та масштабування: Коли необхідно, щоб нові репліки сервісу автоматично приєднувалися до мережі.

Overlay pros and cons:

Pros:

- Підтримка кластерів: Єдиний простір IP-адрес між хостами в кластері;
- Автоматичне тунелювання: Не потрібно вручну налаштовувати маршрути чи VPN — Docker сам створює тунелі;
- [Service Discovery](#): DNS всередині overlay-мережі дозволяє звертатися до сервісів за іменами;
- Вбудоване балансування навантаження між репліками сервісів;
- Безпека: Ізоляція мереж за замовчуванням, шифрування трафіку між хостами (опціонально).

Cons:

- Складність налаштування: Потребує Docker Swarm або окремого ключ-значення сховища (etcd, Consul);
- Затримки: Через тунелювання можуть бути трохи більші затримки, ніж у host чи bridge;
- Залежність від інфраструктури: Не працює у standalone Docker без Swarm або подібної оркестрації;
- Налагодження може бути складним через багаторівневу мережеву архітектуру (VXLAN, iptables, DNS, routing).

Потенційні проблеми з [overlay](#):

- Проблеми з DNS-рішенням, якщо сервіси неправильно зареєстровані;
- Помилки маршрутизації, коли хости не можуть зв'язатися через фізичну мережу;
- Налаштування firewall/iptables: Потрібно відкривати додаткові порти (наприклад, 7946, 4789) між вузлами;
- Складність дебагу: Через віртуальні тунелі складніше діагностувати проблеми з мережею;
- Проблеми сумісності: Застарілі драйвери чи ядра ОС можуть не підтримувати VXLAN.

Головні особливості мережі [overlay](#):

- Дозволяє контейнерам на різних хостах обмінюватися даними так, ніби вони в одній локальній мережі;

- Працює поверх існуючої інфраструктури — без необхідності створювати окрему фізичну мережу;
- Підтримує масштабування та балансування у кластері;
- Використовується для побудови кластеризованих мікросервісів.

Table 3. Порівняння типів мереж в Docker:

Параметр/Мережа	bridge	host	overlay	macvlan	none
Видимість между контейнерами	Да	Только через localhost	Да (между нодами)	Да (в пределах LAN)	Нет
Изоляция	Хорошая	Нет	Отличная	Частичная	Полная
Поддерживает DNS	Да	Нет	Да	Нет (обычно)	Нет
Хост-IP доступ	Только через NAT	Прямой	Только через Ingress	Прямой	Нет
Производительность	Средняя	Высокая	Ниже из-за VXLAN	Высокая	Высокая
Использование в проде	Часто	Ограничено	Swarm/Com pose	Специфика LAN	Никогда
Используется по умолчанию	Да	Нет	Нет	Нет	Нет

3. QUESTIONS

Architecture

- Що буде, якщо Consul втратить 2 вузли в кластері з 5? Які гарантії збережуться? [Answer](#)

Service Discovery Questions

- Що таке Service Discovery і чому він важливий у розподілених системах? [Answer](#)
- Які основні типи Service Discovery ви знаєте? Поясніть різницю між ними. [Answer](#)
- Що таке Client-side discovery? [Answer](#)
- Як працює Client-side discovery? [Answer](#)
- Які переваги та недоліки Client-side discovery? [Answer](#)
- Що таке Server-side discovery? [Answer](#)
- Як працює Server-side discovery? [Answer](#)
- Які переваги та недоліки Server-side discovery? [Answer](#)
- Що таке DNS-based discovery? [Answer](#)
- Як працює DNS-based discovery? [Answer](#)

- Які переваги та недоліки DNS-based discovery? [Answer](#)
- Що таке Key-Value Store-based discovery? [Answer](#)
- Як працює Key-Value Store-based discovery? [Answer](#)
- Які переваги та недоліки Key-Value Store-based discovery? [Answer](#)
- Які переваги та недоліки кожного типу Service Discovery? [Answer](#)
- Де використовується Service Discovery? [Answer](#)
- Що дає Service Discovery у контексті мікросервісної архітектури? [Answer](#)
- Які інструменти для Service Discovery ви знаєте? [Answer](#)
- Які головні компоненти системи Service Discovery? [Answer](#)

Docker Image

- Що таке Docker Image? [Answer](#)
- Яку структуру має Docker Image? [Answer](#)
- З чого складається Docker Image? [Answer](#)
- Що таке Layer в Docker Image? [Answer](#)
- Що таке Layer Cache в Docker Image? [Answer](#)
- Які типи Docker Image існують? [Answer](#)
- Як зберігати секрети в Docker Image? [Answer](#)
- Як створити Docker Image? [Answer](#)
- Як створити Docker Image з Multi-stage build? [Answer](#)
- Як завантажити Docker Image з реєстру? [Answer](#)
- Як опублікувати Docker Image в реєстр? [Answer](#)
- Як додати теги до Docker Image? [Answer](#)
- Як перевірити інформацію про Docker Image? [Answer](#)
- Як видалити Docker Image? [Answer](#)

Dockerfile

- Що таке Dockerfile? [Answer](#)
- Головні інструкції які використовуються в Dockerfile? [Answer](#)
- Що означає FROM в Dockerfile? [Answer](#)
- Використання FROM. [Answer](#)
- Best Practices для FROM. [Answer](#)
- Що означає RUN в Dockerfile? [Answer](#)
- Використання RUN. [Answer](#)
- Best Practices для RUN. [Answer](#)
- Що означає CMD в Dockerfile? [Answer](#)

- Використання **CMD**. [Answer](#)
- Кратко про **CMD**. [Answer](#)
- Що означає **ENTRYPOINT** в Dockerfile? [Answer](#)
- Використання **ENTRYPOINT**. [Answer](#)
- Кратко про **ENTRYPOINT**. [Answer](#)
- В чому різниця між **CMD** та **ENTRYPOINT**. [Answer](#)
- Що означає **COPY** в Dockerfile? [Answer](#)
- Що означає **ADD** в Dockerfile? [Answer](#)
- Використання **ADD**. [Answer](#)
- В чому різниця між **COPY** та **ADD**. [Answer](#)
- Що означає **WORKDIR** в Dockerfile? [Answer](#)
- Що означає **EXPOSE** в Dockerfile? [Answer](#)
- Що означає **ENV** в Dockerfile? [Answer](#)
- Використання **ENV**. [Answer](#)
- Що означає **ARG** в Dockerfile? [Answer](#)
- Використання **ARG**. [Answer](#)
- В чому різниця між **ENV** та **ARG**. [Answer](#)
- Best Practices для **ENV** та **ARG**. [Answer](#)
- Коротко для **ENV** та **ARG**. [Answer](#)
- Що означає **USER** в Dockerfile? [Answer](#)
- Що означає **VOLUME** в Dockerfile? [Answer](#)
- Що означає **LABEL** в Dockerfile? [Answer](#)
- Що означає **HEALTHCHECK** в Dockerfile? [Answer](#)
- Що означає **SHELL** в Dockerfile? [Answer](#)
- Як тестувати Dockerfile? [Answer](#)
- Best Practices при роботі з Dockerfile? [Answer](#)

Docker Volume

- Що таке **VOLUME** в Docker? [Answer](#)
- Чи можна підключати один том до декількох контейнерів? [Answer](#)
- Які типи томів існують в Docker? [Answer](#)
- Що таке Anonymous Volume? [Answer](#)
- Що таке Bind Mount? [Answer](#)
- Що таке Named Volume? [Answer](#)
- Різниця між Bind Mount та Volume? [Answer](#)

- Різниця між Named Volume та Bind Mount [Answer](#)
- Різниця між Named Volume та Bind Mount та Anonymous Volume? [Answer](#)
- Необхідно лі монтувати volume для запису логів додатку? [Answer](#)
- Цілі використання томів в Docker? [Answer](#)
- Які драйвери доступні для томів в Docker? [Answer](#)
- Опції для під час створення Volume? [Answer](#)
- Як видалити невикорсовуємі тома? [Answer](#)
- Як подивитись все доступні Volumes? [Answer](#)

Docker Container

- Що таке Docker Container? [Answer](#)
- Відмінності Docker Container від VM? [Answer](#)
- Чому Docker Container запускається швидше, ніж VM? [Answer](#)
- Как запустить контейнер в фоновом режиме? [Answer](#)
- Як обмежити ресурси контейнера по CPU та пам'яті? [Answer](#)
- Які стани може мати контейнер? [Answer](#)
- Через що контейнер може бути завершеним одразу після запуску? [Answer](#)
- Чим відрізняються `docker stop` від `docker kill`? [Answer](#)
- Флаги `docker run`? [Answer](#)
- Як зменшити час запуску Spring Boot в контейнері? [Answer](#)
- Як шукати проблеми мережі між контейнерами? [Answer](#)
- Як і які обмеження можна задати і якими флагами? [Answer](#)
- Як виконати команду в середині запущеного контейнеру? [Answer](#)
- Як подивитись логі працюючого контейнера? [Answer](#)
- Як зробити порт видимим наружу контейнера? [Answer](#)
- Як встановити змінні середовища при запуску контейнера? [Answer](#)
- Чи можна використовувати файл зі змінними для запуску контейнера? [Answer](#)
- Як обмежити доступ до контейнеру зовні? [Answer](#)
- Як передати файли до контейнеру (або навпаки) без пересборки образу? [Answer](#)
- Як зберегти данні при перезапуску контейнера? [Answer](#)
- Як додати Capabilities є у контейнера? [Answer](#)
- Як перевірити, які Capabilities є у контейнера? [Answer](#)
- Що робить флаг `--privileged` при запуску контейнера? [Answer](#)
- Навіщо використовувати non-root user всередині Docker-контейнера? [Answer](#)
- Як запустити контейнер без мережевого стека? [Answer](#)

- Як забезпечується безпека Docker-контейнерів? [Answer](#)
 - Головні аспекти безпеки Docker-контейнерів? [Answer](#)
 - Яким чином Docker Container забезпечує ізоляцію процесів? [Answer](#)
 - Які Best Practices Docker-контейнерів? [Answer](#)
-

Introduced in different languages:

- [English](#)